Reveal-X

Secure Image Sharing using Visual Cryptography with AI-Based Tamper Detection

By:

Inam Ullah

45521

Zaka Khan

36503

Muhammad Umar

44726

Supervised by:

Sir Yawar Abbas

Lecturer, Faculty of Computing

Faculty of Computing

Riphah International University, Islamabad

Fall 2026

Submitted To

Faculty of Computing,

Riphah International University, Islamabad

As a Partial Fulfillment of the Requirement for the Award of the Degree of

Bachelors of Science in Cyber Security



Faculty of Computing

Riphah International University, Islamabad

Date: December 2026

Final Approval

This is to certify that we have read the report submitted by Inam Ullah (SAP ID 45521), Zaka Khan (SAP ID 36503) and Muhammad Umar (SAP ID 44726), for the partial fulfillment of the requirements for the degree of Bachelor of Science in Cyber Security (BS CYS). It is our judgment that this report is of sufficient standard to warrant its acceptance by Riphah International University, Islamabad for the degree of Bachelor of Science in Cyber Security (BS CYS).

Committee:

1.

_____

Sir Yawar Abbas
(Supervisor)

2.

_____

_____
(Head of Department / Chairman)

Declaration

We hereby declare that this document "Reveal-X: Secure Image Sharing using Visual Cryptography with AI-Based Tamper Detection" neither as a whole nor as a part has been copied out from any source. It is further declared that we have done this project with the accompanied report entirely on the basis of our personal efforts, under the proficient guidance of our teachers especially our supervisor Sir Yawar Abbas. If any part of the system is proved to be copied out from any source or found to be reproduction of any project from anywhere else, we shall stand by the consequences.

_____

Inam Ullah
45521

_____

Zaka Khan
36503

_____

Muhammad Umar
44726

Dedication

To our parents, whose patience outlasted every late night this project cost us, and who never once asked us to choose something easier.

To our supervisor, Sir Yawar Abbas, who insisted that a result we could not reproduce was not a result at all. The measurement discipline in Chapters 4 and 5 is his.

And to the next student who opens this report looking for an honest account of what worked and what did not: Section 5.6 and Section 6.3 are written for you.

Acknowledgement

First of all we are obliged to Allah Almighty the Merciful, the Beneficent and the source of all Knowledge, for granting us the courage and knowledge to complete this Project.

We are deeply grateful to our supervisor, Sir Yawar Abbas, Lecturer at the Faculty of Computing, for his guidance across both phases of this project. His insistence that every claim be backed by a measurement we could re-run shaped the evaluation methodology of this report more than any other single influence.

We thank the Faculty of Computing at Riphah International University, Islamabad, for the laboratory access and the structured review process that kept the project on schedule. We also thank the Project Coordinator for the milestone reviews, which caught several problems early enough to be fixed cheaply.

Finally, we thank our families and our classmates, who tested the application, broke it in ways we had not anticipated, and reported the results honestly.

_____

Inam Ullah
45521

_____

Zaka Khan
36503

_____

Muhammad Umar
44726

Abstract

Conventional messaging platforms require their users to trust the server with the images they share. The server receives a complete, viewable copy, and a compromise of that server, an insider, or a compelled disclosure order exposes it. Reveal-X is a real-time secure messaging application that removes that requirement for image content, and then verifies that what the recipient receives is what was sent.

The system implements a (2,2) XOR visual cryptography scheme. Each shared image is split into two shares: Share 1 is a cryptographically random mask drawn from the operating system CSPRNG and is stored on the server, while Share 2 is the exclusive-or of the image with that mask and is delivered live to the recipient and never written to server storage. Either share alone is a one-time pad and reveals nothing beyond the image dimensions. Measured over six test images, reconstruction is bit-exact in every case: mean squared error of exactly zero and a structural similarity index of exactly 1.0000. Chi-square, adjacent-pixel correlation and Shannon entropy tests confirm that the shares are statistically indistinguishable from uniform random data, with entropy at the 8.000 bits per pixel maximum and horizontal correlation falling from 0.9788 on a natural image to 0.0035 on its share.

Integrity is enforced in two independent layers. A keyed HMAC-SHA256 tag over the stored bytes detects every one of 420 simulated attacks across seven attack classes. A RandomForest classifier over thirteen statistical features, trained on 240 paired samples and evaluated on an independent set of 840, reaches 92.38% accuracy with 99.17% precision, localises damage to a 16x16 block heatmap, and runs identically on the server under scikit-learn and in the recipient's browser under ONNX Runtime, with zero verdict disagreements over 200 samples. The two layers fail in different places, and the report documents this rather than concealing it: the classifier does not detect random-noise injection at any strength tested, because replacing uniform random pixels with fresh uniform random pixels produces the feature vector of a clean share. Eight classifier families were compared and all converge on the same ceiling, establishing that the limitation lies in the feature representation rather than in the learner. The HMAC layer catches that attack in 60 of 60 trials.

Text and voice messages are end-to-end encrypted using ECDH on NIST P-256 with HKDF-SHA256 derivation and AES-256-GCM, with non-extractable private keys held in IndexedDB. Authentication uses Argon2id password hashing, an optional RFC 6238 time-based second factor, a three-rank authorisation model, and throttling on both the account and the network address. The implementation comprises approximately 4,700 lines of Python and 5,800 lines of JavaScript, verified by 416 server-side and 69 browser-side automated tests. One stated objective, deep-learning image enhancement, was deliberately not delivered; Section 5.6 gives the measured reason and a costed route to completing it.

Table of Contents

Final Approval IV

Declaration V

Dedication VI

Acknowledgement VII

Abstract VIII

Table of Contents X

List of Figures XIV

List of Tables XVI

Chapter 1: Introduction 2

1.1 Introduction 2

1.2 Opportunity and Stakeholders 2

1.3 Motivations and Challenges 3

1.4 Significance of the Study 4

1.5 Goals and Objectives 5

1.6 Scope of the Project 5

1.7 Chapter Summary 6

Chapter 2: Literature and Market Survey 9

2.1 Introduction 9

2.2 Literature Review and Technologies Overview 9

2.2.1 Visual cryptography schemes 9

2.2.2 Conventional image encryption 10

2.2.3 Steganography and digital watermarking 10

2.2.4 Image forensics and tamper detection 10

2.2.5 AI-based image enhancement 11

2.2.6 Secure messaging products 11

2.3 Comparative Analysis 11
2.4 Research Gaps 13
2.5 Problem Statement 13
2.6 Chapter Summary 14
Chapter 3: Requirements and System Design 16
3.1 Introduction 16
3.2 System Architecture 16
3.3 Functional Requirements 18
3.4 Non-Functional Requirements 19
3.5 Design Diagrams 20
3.5.1 Use case model 20
3.5.2 Visual cryptography data flow 22
3.5.3 Database design 22
3.6 Hardware and Software Requirements 23
3.7 Threat Scenarios 24
3.8 Threat Modeling Techniques 24
3.9 Threat Resistance Model 25
3.10 Chapter Summary 26
Chapter 4: Proposed Solution 28
4.1 Introduction 28
4.2 Proposed Model 28
4.2.1 The visual cryptography scheme 28
4.2.2 Two-layer integrity 29
4.2.3 The detector 29
4.3 Data Collection 30
4.4 Data Pre-Processing 32
4.5 Tools and Techniques 32

4.6 Evaluation Metrics 33
4.7 Chapter Summary 34
Chapter 5: Implementation and Testing 36
5.1 Security Properties Testing 36
5.2 System Setup 37
5.3 System Integration 39
5.4 Penetration Testing and Vulnerability Assessment 39
5.5 Test Cases 41
5.5.1 Test metrics 43
5.6 Results and Discussion 44
5.6.1 Reconstruction fidelity 44
5.6.2 Statistical security of the shares 45
5.6.3 Tamper detection results 49
5.6.4 The noise blind spot 53
5.6.5 Two-layer integrity 55
5.6.6 Operating point and model selection 56
5.6.7 Objective 9: not delivered, and why 58
5.6.8 Browser and server parity 60
5.6.9 Performance 61
5.7 Best Practices and Coding Standards 62
5.7.1 Code validation 62
5.7.2 Development practices and standards 63
5.8 Chapter Summary 64
Chapter 6: Conclusion and Future Work 66
6.1 Introduction 66
6.2 Achievements and Improvements 66
6.3 Critical Review 67

6.4 Future Recommendations 68
6.5 Chapter Summary 69
References 70
Appendix A: Installation and Interface Reference 75
A.1 Installation 75
A.2 HTTP routes 75
A.3 Socket.IO events 76
A.4 Repository layout 76

List of Figures

Figure 3.1: Layered architecture of Reveal-X. The shaded layer is the security and machine-learning core added in the second phase of the project. 17
Figure 3.2: Component and deployment view. Module names are those in the repository. 18
Figure 3.3: Use case model. Ranks inherit downward; an administrator may act only on an account strictly below its own. 21
Figure 3.4: Visual cryptography data flow. The server holds the plaintext image only transiently, at step 2 22
Figure 5.1: The end-to-end encryption handshake and message path, including the image exception. 38
Figure 5.2: The (2,2) XOR pipeline on a 256x256 test image: the secret, the two shares, and the recovered image 44
Figure 5.3: Grey-level histograms on a logarithmic count axis. The original and the recovery are strongly peaked; both shares are flat at the uniform expectation. 46
Figure 5.4: Adjacent horizontal pixel pairs, 4000 samples. Natural images lie on the diagonal; a share fills the square uniformly. 46
Figure 5.5: Shannon entropy of each array across the six test images. Both shares sit at the 8.000 bits per pixel maximum regardless of the content they conceal. 47
Figure 5.6: Shares do not compress. PNG output size and compression ratio for three originals and their Share 1 48
Figure 5.7: Confusion matrix at the shipped 0.50 threshold, and the distribution of predicted probabilities by true class. 49
Figure 5.8: ROC and precision-recall curves on the independent set. The dotted line marks the recall ceiling imposed by the noise blind spot. 49
Figure 5.9: Detection rate by attack type at two thresholds, against the HMAC verification rate on the same attacks 50
Figure 5.10: Gini importance of the thirteen features in the shipped RandomForest. 52
Figure 5.11: Left: the entropy distribution of clean and tampered shares. Right: effect size per feature, measured as Cohen's d 52

Figure 5.12: Noise attack across ten strengths: detection by the classifier and by HMAC, with the fraction of pixels the attack actually changed. 53
Figure 5.13: Coverage matrix. Neither layer detects every attack; the union of the two does. 55
Figure 5.14: One block attack end to end: the clean share, the tampered share, the suspicion heatmap, and the visibly corrupted reconstruction. 56
Figure 5.15: Recall, precision, F1 and false positive rate against the decision threshold. . 56
Figure 5.16: Eight classifiers on identical features. All cluster at the ceiling imposed by the blind spot. 57
Figure 5.17: Agreement between the scikit-learn model on the server and the ONNX model in the browser over 200 samples. 60
Figure 5.18: Per-stage latency against image size, and the composition of the total detection cost. 61
Figure 5.19: Composition of the automated test suite by area. 62

List of Tables

Table 1.1: Project objectives and their delivery status. Objective 9 was deliberately not delivered; Section 5.6 gives the reason. 5

Table 2.1: Literature review summary. The final column records what each work contributes to the present system. 11

Table 2.2: Reveal-X against the principal visual cryptography schemes. No surveyed scheme carries an integrity check. 12

Table 2.3: Reveal-X against production messaging systems. 12

Table 3.1: Functional requirements, with the interface that implements each and the test module that verifies it. 18

Table 3.2: Non-functional requirements, their acceptance criteria and the measured outcome. 19

Table 3.3: Use case specification: Share an image. 21

Table 3.4: Database schema. Columns marked with an asterisk carry direct security significance. 23

Table 3.5: Hardware and software requirements. 23

Table 3.6: STRIDE analysis against the elements of the data-flow diagram. 25

Table 3.7: Threat resistance model. Three adversaries are not defended against, and are stated as plainly as the rest. 26

Table 4.1: The thirteen-feature vector, its interpretation, and the tampering signal each feature is intended to capture. 30

Table 4.2: The seven simulated attacks used for training and evaluation, with the real-world case each stands in for. 31

Table 4.3: The three datasets. The evaluation set is fourteen times the size of the internal test split and was never seen during fitting. 31

Table 4.4: Tools and techniques, with the role each plays. 32

Table 4.5: Evaluation metrics and their acceptance criteria. 33

Table 5.1: Security-relevant configuration. 37

Table 5.2: Cryptographic parameters of the end-to-end encrypted channel. 37

Table 5.3: The automated test suite by module. The distribution reflects where a defect would be most costly. 41

Table 5.4: Test case TC-01: Share 2 is never written to server storage. 41
Table 5.5: Test case TC-02: A tampered share is detected and localised before reconstruction. 42
Table 5.6: Test case TC-03: Share 1 can be fetched exactly once. 42
Table 5.7: Test case TC-04: The TOTP attempt limit survives reconnection. 43
Table 5.8: Test metrics over the complete suite. 44
Table 5.9: Reconstruction fidelity over six test images. Every reconstruction is bit-exact. 44
Table 5.10: Statistical profile of the originals, the shares and the recovery for two representative test images. 48
Table 5.11: Storage cost of a share. A share is incompressible, so storing it costs the full uncompressed size of the image. 48
Table 5.12: Headline detection performance on the independent set (n = 840), at three decision thresholds. 49
Table 5.13: Per-attack detection on the independent set. Six attacks are detected essentially without error; one is not detected at all. 50
Table 5.14: The eight most important features, with effect size and significance measured independently of the model on the evaluation set. 52
Table 5.15: The noise attack swept from 5% to 75% strength, 40 trials per level. The attack is real at every level and invisible to the classifier at every level. 53
Table 5.16: The two integrity layers compared on identical attacks. 55
Table 5.17: Operating points. Raising the threshold from 0.50 to 0.60 removes every false positive without costing a single detection. 56
Table 5.18: Eight classifiers trained on identical features and the same split, evaluated on the same independent set, with the rule-based fallback for reference. 57
Table 5.19: Server-to-browser agreement. The model and the feature extractor were verified separately. 60
Table 5.20: Median latency in milliseconds by stage and image size. The cryptographic operations are negligible; the heatmap dominates. 61
Table 5.21: Size of the implementation by component. 63
Table A.1: The nine HTTP routes. 75
Table A.2: Socket.IO event inventory. 76
Table A.3: Principal source files. 76

Chapter 1:

Introduction

Chapter 1: Introduction

1.1 Introduction

Digital images now carry a large share of the sensitive information that moves between people and institutions. Medical scans travel between clinics, identity documents are submitted to banks and government portals, photographs of evidence pass between legal teams, and screenshots of confidential records are shared inside organisations every day. In each case two properties matter and are usually conflated: whether anyone other than the intended recipient can see the image, and whether the image the recipient opens is the image that was sent.

Mainstream messaging platforms address the first property well and the second barely at all. Transport encryption and at-rest encryption protect a file while it moves and while it sits on disk, but once a server has stored an object there is no per-object answer to the question "has this specific stored image been altered, and if so where?" Integrity is treated as a property of the channel rather than of the artefact.

Reveal-X approaches both properties from a different starting point. Instead of encrypting an image and asking the recipient to trust the server that holds it, the system splits the image into two shares using visual cryptography, so that nothing the server persists is sufficient to reconstruct it. It then attaches two independent integrity checks to the stored share — a keyed cryptographic tag and a machine-learned tamper detector — and runs both before the recipient is allowed to reconstruct. The result is a working real-time chat application in which image confidentiality rests on an information-theoretic argument rather than a computational one, and in which integrity is verified at the point of use.

This chapter states the opportunity the project addresses, who it is for, the motivations and challenges behind it, why the work is significant, and the goals, objectives and scope against which the rest of the report should be judged.

1.2 Opportunity and Stakeholders

The opportunity is a gap between two mature bodies of work that have not been combined in a usable product. Visual cryptography has been studied since 1994 and is well understood theoretically, but almost every implementation is an isolated prototype: a command-line tool operating on static image files, with no communication layer, no integrity verification and no user interface. Secure messaging, meanwhile, is a mature

product category in which no mainstream system offers per-object integrity verification of stored media. Reveal-X occupies the intersection.

The stakeholders for a system of this kind fall into four groups.

- **Clinical and healthcare users.** Medical images are routinely shared between practitioners over general purpose messaging tools. A scheme in which the relaying server cannot view the image, and in which the recipient is told before opening it whether the stored copy was altered, addresses a real and currently unmet confidentiality requirement.
- **Legal and investigative users.** Photographic evidence has value only if its integrity can be demonstrated. The per-share HMAC tag and the localising tamper detector together provide an artefact-level answer rather than a channel-level one.
- **Organisations handling identity documents.** Any process that accepts a photographed identity document is exposed both to server-side disclosure and to submitted images that have been edited. The two concerns are usually handled by separate systems; here they are handled by one.
- **The academic and research community.** The project contributes a measured, reproducible evaluation of what a lightweight statistical tamper detector can and cannot do on visual cryptography shares, including a negative result that generalises beyond this implementation.

The immediate stakeholders of the project itself are the project team, the supervisor and the examining committee, for whom the defensibility of each claim matters as much as the functionality it describes.

1.3 Motivations and Challenges

Three observations motivated the work. The first is that trust in the server is the weakest link in conventional image sharing, and that it is avoidable rather than inherent. The second is that integrity and confidentiality are commonly bundled together when they are separable properties with different failure modes: an attacker who cannot read an image may still be able to corrupt it. The third is that visual cryptography research has largely stopped at the scheme, leaving the question of how a share should be stored, verified and delivered in a live system unanswered.

Four challenges followed directly from those choices.

- **Storage cost.** A visual cryptography share is incompressible by construction. Measured with PNG, a share compresses by a factor of 0.993 — that is, not at all — whereas the gradient test image compresses 142-fold. Storing a share therefore costs the full uncompressed size of the image, a 143-fold increase in the worst case measured. This drove the one-hour message expiry.
- **Verifying a random-looking artefact.** A tamper detector normally looks for anomalies against structured content. A valid share is uniform random noise, which inverts the usual asymmetry: the expected profile is known exactly, which helps, but an attack that injects randomness is invisible, which does not. Section 5.6 measures this precisely.
- **Reproducing the detector in two runtimes.** For the recipient's browser to form an independent opinion, it must reproduce thirteen statistical features and a RandomForest from raw pixels, in JavaScript, and agree with the server. Numerical agreement across float32 and float64 had to be verified rather than assumed.
- **Honest scoping.** One objective inherited from the first phase of the project turned out to rest on a false premise. Deciding not to deliver it, and documenting the measurement that justified that decision, was more difficult than implementing it would have been.

1.4 Significance of the Study

The significance of this project is not a new cryptographic construction. The (2,2) XOR visual cryptography scheme is established work, and the implementation follows it. The contribution lies in three places, each narrow and each measured.

First, the project demonstrates that information-theoretically secure image sharing can be embedded in a real-time messaging application at acceptable cost. The cryptographic core is not the expensive part: at 256x256 pixels, generating both shares takes 0.015 ms, reconstruction 0.003 ms and the integrity tag 0.033 ms. The common assumption that visual cryptography is too costly for interactive use is not supported by measurement.

Second, it establishes empirically that a keyed cryptographic tag and a learned statistical detector are complementary rather than redundant on this problem. Each catches attacks the other misses, and the report quantifies both the overlap and the gap instead of asserting that two layers are better than one.

Third, it contributes a negative result with a structural explanation. The classifier cannot detect random-noise injection at any strength between 5% and 75%, and eight different classifier families trained on the same features all converge on the same ceiling. The limitation is in the representation, not the learner. A system relying on machine-learned integrity checking alone would be trivially exploitable by an attacker who knew this, which is a result worth stating plainly.

1.5 Goals and Objectives

The primary goal of the project is to design, implement and evaluate a real-time secure messaging application in which shared images are protected by visual cryptography and verified by two independent integrity layers, with every security and performance claim supported by a measurement that can be re-run against the repository.

The specific objectives, carried across both phases of the project, are listed in Table 1.1 together with their delivery status. The evidence for each is given in the chapter indicated.

Table 1.1: Project objectives and their delivery status. Objective 9 was deliberately not delivered; Section 5.6 gives the reason.

#	Objective	Status	Evidence
1	Implement a (2,2) visual cryptography scheme for share generation and exact reconstruction	Delivered	§4.2, §5.6
2	Integrate a real-time messaging engine with presence, typing indicators and read receipts	Delivered	§3.3, §5.3
3	Implement multi-layer cross-site scripting prevention	Delivered	§5.1
4	Apply memory-hard password hashing resistant to GPU attack	Delivered	§5.1 (Argon2id)
5	Provide a responsive, accessible user interface	Delivered	§5.7 (WCAG 2.1 AAA contrast)
6	Package the application as an installable Progressive Web App	Delivered	§5.3
7	Train and integrate machine-learned tamper detection for shares	Delivered	§4.2–4.6, §5.6
8	Complete end-to-end encryption of messages with AES-GCM	Delivered	§5.2, Table 5.3
9	Add a denoising CNN and ESRGAN super-resolution enhancement	Not delivered	§5.6.7, with justification
10	Evaluate accuracy, reconstruction fidelity and latency	Delivered	§4.6, §5.6

1.6 Scope of the Project

The boundaries of the work are stated here so that the evaluation in Chapter 5 is read against the right expectations.

Within scope:

- A (2,2) XOR visual cryptography scheme operating on grayscale images, with server-side share generation and recipient-side reconstruction.
- Per-share integrity verification by HMAC-SHA256 and by a thirteen-feature RandomForest classifier, the latter running both on the server and in the recipient's browser.
- End-to-end encryption of text and voice messages using ECDH P-256, HKDF-SHA256 and AES-256-GCM.
- Authentication with Argon2id password hashing, an optional TOTP second factor, optional Firebase-backed Google sign-in, a three-rank authorisation model and brute-force throttling.
- A real-time one-to-one messaging interface with presence, typing indicators, read receipts, reactions, forwarding, replies and voice notes, delivered as an installable Progressive Web App.
- A separate evaluation dashboard that reproduces the measurements in this report without requiring an account.

Outside scope:

- Colour visual cryptography. The scheme operates on a single channel; colour images are converted to grayscale before splitting.
- Group messaging. All conversations are strictly one-to-one.
- Forward secrecy. ECDH key pairs are long-lived; a ratcheting construction was not implemented.
- End-to-end encryption of images. The split is performed on the server, which therefore sees the plaintext image at that moment. This boundary is stated explicitly in Section 3.7 and Section 6.3.
- Deep-learning image enhancement, for the reason given in Section 5.6.7.
- Formal penetration testing by an independent third party. The security testing reported in Section 5.1 and Section 5.4 was performed by the project team.

1.7 Chapter Summary

This chapter has set out the problem Reveal-X addresses — that conventional image sharing requires trust in the server for confidentiality and offers no artefact-level integrity guarantee at all — identified the stakeholders, and stated the goals, objectives and scope of

the work. Objective 9 is flagged here as not delivered so that the reader encounters that fact at the outset rather than at the end.

The remainder of the report is organised as follows. Chapter 2 surveys the visual cryptography, image encryption and image forensics literature alongside the production messaging products the system is compared against, and derives the research gap and problem statement from that survey. Chapter 3 specifies the functional and non-functional requirements, presents the system architecture and design diagrams, states the hardware and software requirements, and develops the threat model using STRIDE. Chapter 4 describes the proposed solution: the visual cryptography model, how the evaluation data was collected and pre-processed, the tools and techniques used, and the metrics by which the system is judged. Chapter 5 reports the implementation, the system setup and integration, the security properties testing, the test cases and metrics, and the measured results including the classifier's blind spot. Chapter 6 concludes with the achievements, a critical review of what the project did not get right, and recommendations for future work.

Chapter 2:

Literature and Market Survey

Chapter 2: Literature and Market Survey

2.1 Introduction

This chapter surveys the work Reveal-X builds on and the products it competes with. It covers four bodies of literature — visual cryptography, conventional image encryption, steganography and watermarking, and image forensics — together with the secure messaging products that define current practice. The purpose is not completeness but positioning: each section ends by stating what the work surveyed does and does not provide for the problem in hand, and the chapter closes by deriving the research gap and the problem statement from those findings rather than asserting them.

2.2 Literature Review and Technologies Overview

2.2.1 Visual cryptography schemes

Visual cryptography was introduced by Naor and Shamir in 1994 [1]. Their (k,n) threshold construction encodes a binary secret image into n shares such that any k shares reveal the secret when stacked as printed transparencies, while any k-1 shares reveal nothing. The decoding step requires no computation: the human visual system performs it. That property is the scheme's distinguishing advantage and the source of its two structural costs. Each pixel expands into a block of subpixels, so the shares are two to four times the size of the original, and the recovered image has at most half the contrast of the input. Both costs are unavoidable consequences of decoding by eye.

The construction is the two-party specialisation of threshold secret sharing, introduced independently by Shamir [2] and Blakley [3]. Ateniese et al. [4] generalised visual cryptography to arbitrary access structures, at the cost of expansion that grows with the structure. Zhou et al. [6] and Wang et al. [7] addressed grayscale reproduction through halftoning and error diffusion respectively, achieving richer tonal range while retaining pixel expansion of four or more and introducing halftone artefacts of their own.

The XOR-based family described by Tuyls et al. [5] replaces stacking with a computed exclusive-or. Because decoding is performed by software rather than by eye, pixel expansion disappears and recovery becomes exact. This is the branch Reveal-X implements, and the reasoning is given in Section 4.2: the contrast penalty of the classical construction buys a property this application does not need.

2.2.2 Conventional image encryption

AES in an authenticated mode [13], [14] and RSA remain the dominant approach to protecting image data in commercial systems, and chaotic-map schemes are a recurring alternative in the literature. AES-256-GCM provides strong ciphertext indistinguishability and is used in this project for text and voice content. For image content, however, conventional encryption has a property that visual cryptography does not: the ciphertext and the key together are held, at some point, by whatever party performs decryption, and the security argument is computational. Visual cryptography's guarantee is information-theoretic, resting on the one-time pad result of Shannon [9], and therefore unaffected by future advances in computing power. RSA is in any case unsuitable for bulk image data, and chaotic-map schemes have weaker formal security arguments than either alternative.

2.2.3 Steganography and digital watermarking

Steganography conceals the existence of a message inside a carrier, and digital watermarking embeds ownership or authentication metadata. Both are valuable for their own purposes and neither solves the problem addressed here. A steganographic payload is recoverable by an adversary who knows the embedding algorithm, so it provides concealment rather than confidentiality. A watermark can be stripped or overwritten, and its absence is indistinguishable from an image that never carried one. Reveal-X uses neither; confidentiality comes from the secret-sharing construction and integrity from a keyed tag whose forgery is computationally infeasible without the key.

2.2.4 Image forensics and tamper detection

Modern image forensics operates on natural images, where the task is to detect a splice or manipulation against a complex, highly structured background. Fridrich and Kodovsky [27] established the rich-model approach, using high-dimensional statistical feature sets. Bayar and Stamm [28] replaced hand-designed features with a learned convolutional front-end, and Cozzolino and Verdoliva [29] extended this to camera-model fingerprinting.

The problem in this project differs from that literature in two ways that pull in opposite directions, and both are consequential. It is easier, because a valid share is uniform random noise: the expected statistical profile is known exactly in advance, so a small feature set suffices and the null hypothesis is fully specified. It is harder, because on a uniform random share, injected randomness is indistinguishable from the content itself — the artefact that is most conspicuous on a natural image becomes invisible here. Section 5.6.4 measures the consequence.

2.2.5 AI-based image enhancement

Denoising networks such as DnCNN [30] and super-resolution models such as SRGAN [31] and ESRGAN [32] have been applied to visual cryptography reconstructions in the literature [40], motivated by the lossy recovery of classical stacking-based schemes. Section 5.6.7 explains why that motivation does not transfer to an XOR-based scheme whose reconstruction is bit-exact, and why applying generative super-resolution to a security artefact is actively undesirable.

2.2.6 Secure messaging products

Signal and WhatsApp provide end-to-end encryption by default with forward secrecy through the Double Ratchet [19] and an X3DH handshake [20]. Telegram encrypts end-to-end only in secret chats; its default cloud chats are readable by the server. All three protect media in transit and at rest, and none offers a per-object integrity check that a recipient can evaluate independently once the object has been stored. That absence is the specific gap this project addresses, and Section 2.3 quantifies both the gap and the places where Reveal-X is weaker than these products.

2.3 Comparative Analysis

Table 2.1 summarises the principal works surveyed above against the criteria that matter for this project. The columns are chosen to expose the specific properties Reveal-X depends on rather than to describe each work in general terms.

Table 2.1: Literature review summary. The final column records what each work contributes to the present system.

No.	Work, reference	Year	Contribution	Limitation for this problem	Used here
1	Naor & Shamir [1]	1994	Founding (k,n) visual cryptography; decoding by human vision	Pixel expansion 2x–4x; contrast halved; binary images only	Conceptual basis
2	Shamir [2]; Blakley [3]	1979	Threshold secret sharing	Not specialised to images	Theoretical grounding
3	Ateniese et al. [4]	1996	General access structures	Expansion grows with the structure	Not required at (2,2)
4	Tuyls et al. [5]	2005	XOR-based VC: no expansion, exact recovery	Requires computational decoding	Scheme implemented
5	Zhou et al. [6]; Wang et al. [7]	2006, 2009	Halftone VC for grayscale	Expansion ≥4x; halftone artefacts	Superseded by XOR
6	Shannon [8], [9]	1948, 1949	Entropy; perfect secrecy of the one-time pad	Requires a fresh, uniform pad	Security argument; entropy metric
7	Krawczyk et al. [11]	1997	HMAC keyed authentication	Binary verdict; no localisation	Integrity layer 1
8	Fridrich & Kodovsky [27]	2012	Rich statistical models for forensics	High dimensionality; aimed at natural images	Feature-design precedent
9	Bayar & Stamm [28]; Cozzolino & Verdoliva [29]	2016, 2020	Learned CNN forensic front-ends	Assumes structured content; heavy for in-browser use	Not adopted; rationale §2.2.4

No.	Work, reference	Year	Contribution	Limitation for this problem	Used here
10	Breiman [21]	2001	Random forests	Needs informative features	Integrity layer 2
11	Zhang et al. [30]; Wang et al. [32]	2017, 2018	DnCNN denoising; ESRGAN super-resolution	Assumes lossy input; GAN detail is synthesised	Rejected; §5.6.7
12	Marlinspike & Perrin [19], [20]	2016	Double Ratchet and X3DH	Substantial protocol complexity	Not implemented; §6.4

Table 2.2 positions the implemented scheme against the visual cryptography literature specifically. The contribution is visible in the last two columns: the scheme itself is established work, and what the project adds is the integrity layer above it.

Table 2.2: Reveal-X against the principal visual cryptography schemes. No surveyed scheme carries an integrity check.

Scheme	Decoding	Pixel expansion	Recovery quality	Integrity check	Grayscale
Naor–Shamir (2,2) [1]	Human visual, stacking	2x to 4x	Lossy; contrast halved	None	Binary only
General access structures [4]	Human visual, stacking	Grows with structure	Lossy	None	Binary only
Halftone VC [6], [7]	Human visual, stacking	4x or more	Lossy; halftone artefacts	None	Via halftoning
XOR-based VC [5]	Computed XOR	None	Exact	None	Yes
Reveal-X (this work)	Computed XOR	None	Exact (MSE 0, SSIM 1.0000)	HMAC-SHA256 and a 13-feature classifier	Yes

Table 2.3 compares the system against production messaging platforms. It is included to be explicit about where this project is weaker, not only where it differs. Two rows go the wrong way for Reveal-X and are highlighted in the discussion that follows.

Table 2.3: Reveal-X against production messaging systems.

Property	Signal / WhatsApp	Telegram (default)	Reveal-X
Text end-to-end encrypted	Yes	No — cloud chats are server-readable	Yes, AES-GCM-256
Forward secrecy	Yes, Double Ratchet [19]	In secret chats only	No — long-lived ECDH keys
Image confidentiality from the server	Yes	No in cloud chats	Partial — split server-side; no stored share suffices
Stored-media integrity check	Transport-level only	Transport-level only	Per-share HMAC-SHA256 plus an ML classifier
Tamper localisation	None	None	Yes, 16x16 block heatmap
Second factor	Registration PIN	Optional password	TOTP, RFC 6238 [16]
Independently auditable	Partly, published protocol	Partly	Yes — open implementation with a published threat model

Two comparisons deserve emphasis. Reveal-X has no forward secrecy, a property Signal has provided for over a decade, so compromise of a long-lived private key exposes previously captured messages in that conversation. And images are not end-to-end encrypted, whereas in Signal and WhatsApp they are; the server performs the visual cryptography split and therefore sees the plaintext image at that moment. A final-year project claiming parity with those systems would be misrepresenting itself.

Where the system does something those products do not is stored-media integrity. Mainstream messengers protect media with transport and storage encryption, but once a file is stored there is no per-object check that answers whether that specific object has been altered and where. Reveal-X answers both parts: the HMAC answers whether, and the classifier answers where. That is the contribution, and it is stated at its true size.

2.4 Research Gaps

The survey identifies four gaps that together define the work.

- **Gap 1 — visual cryptography is not deployed.** Published schemes are evaluated as algorithms on static files. The questions a deployment raises — how a share is stored, how its integrity is established, how the second share reaches the recipient without being persisted, what the storage cost actually is — are largely unaddressed.
- **Gap 2 — no integrity layer on shares.** None of the schemes in Table 2.2 carries an integrity check of any kind. An adversary who modifies a share in transit or at rest cannot be detected by the scheme itself, and the recipient reconstructs a corrupted image with no indication that anything is wrong.
- **Gap 3 — forensic methods are not characterised on random-looking artefacts.** The forensics literature targets natural images. How a statistical detector behaves when the clean class is uniform random noise, and which attacks become invisible under that inversion, is not established.
- **Gap 4 — no per-object integrity verification in production messengers.** As Table 2.3 shows, current products treat integrity as a property of the channel. A recipient cannot independently verify a stored object.

2.5 Problem Statement

Users who share sensitive images through conventional messaging systems must trust the relaying server with a complete, viewable copy of the image, and have no means of verifying that the stored copy they later receive is the one that was sent. Visual

cryptography can eliminate the first requirement, because no single share reveals anything about the original, but published schemes provide no integrity guarantee, have not been integrated into real-time communication systems, and have not been characterised when a statistical tamper detector is applied to shares whose clean state is itself uniform random noise.

The problem this project addresses is therefore to design, build and measure a real-time messaging system that:

- splits every shared image so that nothing the server persists is sufficient to reconstruct it;
- attaches to each stored share an integrity mechanism that detects modification and localises it, and establishes by measurement which attacks that mechanism does and does not catch;
- delivers both properties at a latency acceptable for interactive use; and
- states its own limitations with the same precision as its results.

2.6 Chapter Summary

This chapter has surveyed visual cryptography from Naor and Shamir through the XOR-based family the project implements, conventional image encryption, steganography and watermarking, the image forensics literature, and the production messaging systems the result is compared against. The comparative analysis in Tables 2.1 to 2.3 shows that no surveyed visual cryptography scheme carries an integrity check, and that no production messenger offers per-object integrity verification of stored media, while also recording two properties — forward secrecy and end-to-end encrypted images — on which those products are stronger than this system. Four research gaps were derived from the survey and consolidated into the problem statement in Section 2.5. Chapter 3 turns that problem statement into requirements, a system design and a threat model.

Chapter 3:

Requirements and System Design

Chapter 3: Requirements and System Design

3.1 Introduction

This chapter turns the problem statement of Section 2.5 into a buildable specification. It states the functional and non-functional requirements, presents the architecture at two levels of detail, gives the design diagrams, records the hardware and software the system needs, and develops the threat model that the security testing in Chapter 5 is measured against.

One design decision shapes everything that follows and is stated here rather than left to be discovered. The visual cryptography split is performed on the server, not in the browser. An earlier version of this project documented a client-side split; the implemented system calls the share generator inside the socket handler for an uploaded image. The consequence is a narrower confidentiality guarantee than a client-side split would give, and it is carried consistently through the architecture in Section 3.2, the data flow in Section 3.5, the threat model in Section 3.9, and the limitations in Section 6.3.

3.2 System Architecture

Reveal-X follows a three-tier architecture: a browser client, a Python application tier built on Flask and Flask-SocketIO, and a persistence tier holding the relational database and the stored shares. Figure 3.1 shows the layered view, with the security and machine-learning core highlighted.

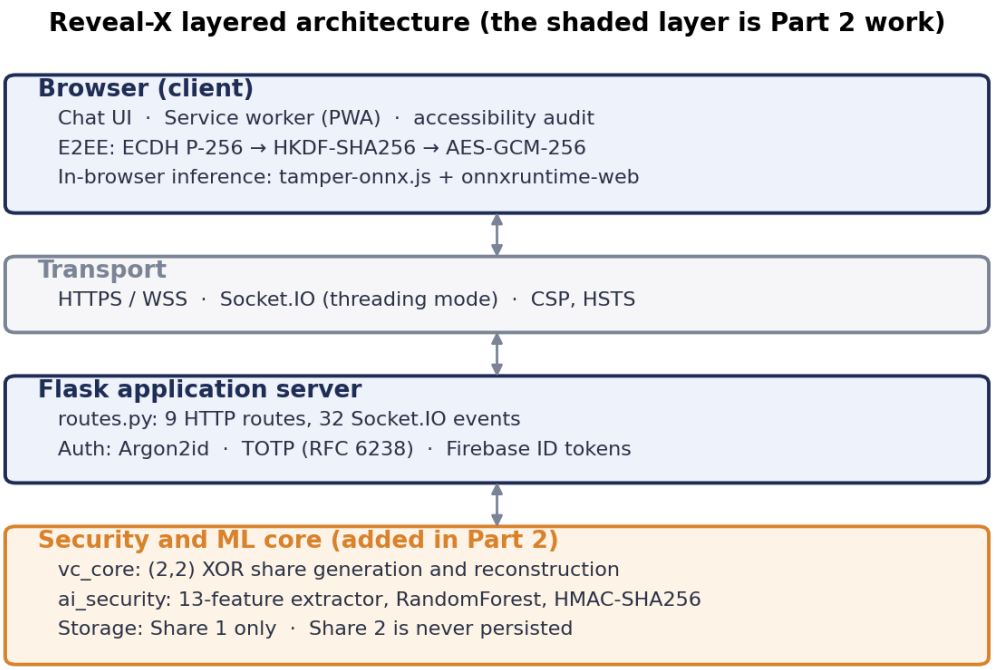


Figure 3.1: Layered architecture of Reveal-X. The shaded layer is the security and machine-learning core added in the second phase of the project.

The browser carries more responsibility than in a conventional web application. It derives its own content encryption key, holds the private half of that key in a form no script can read back, and runs the tamper detection model locally so that it can form an opinion about a received share independently of the server that supplied it. Figure 3.2 gives the component view with the actual module names.

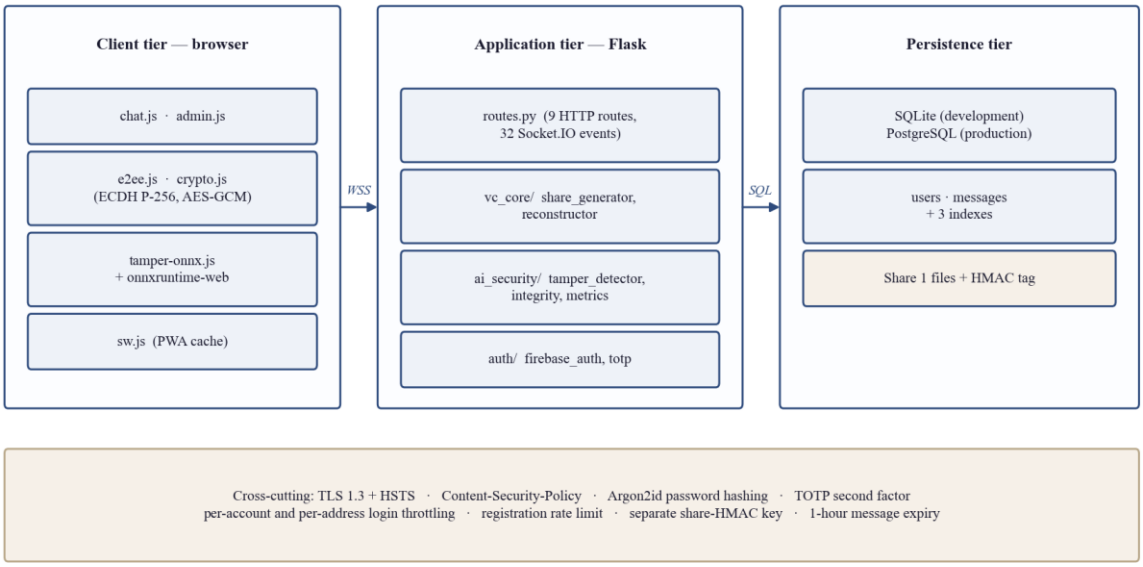


Figure 3.2: Component and deployment view. Module names are those in the repository.

One invariant holds across every layer and is the single most important property of the system: Share 2 is never written to server storage. It is generated in memory, transmitted to the recipient over the live socket, and discarded. A constant in the storage layer, FORBIDDEN_EXTRA_KEYS, refuses any attempt to persist it, and the test suite asserts that refusal directly, so a later change cannot weaken the property by accident.

3.3 Functional Requirements

The functional requirements are listed in Table 3.1. Each is traceable to an implemented handler — the system exposes nine HTTP routes and thirty-two Socket.IO events — and to the test module that holds it in place. Priority is recorded as Essential, Expected or Desirable.

Table 3.1: Functional requirements, with the interface that implements each and the test module that verifies it.

ID	Requirement	Priority	Verified by
FR-1	A visitor shall be able to request an account; no account is usable until an administrator approves it	Essential	test_admin.py
FR-2	A user shall authenticate with a username and password hashed with Argon2id	Essential	test_auth.py
FR-3	A user shall optionally authenticate through Google, verified server-side by Firebase ID token	Desirable	test_auth.py
FR-4	A user shall optionally enrol a TOTP second factor; no session is issued until a valid code is supplied	Expected	test_auth.py
FR-5	The system shall throttle failed sign-in attempts per account and per network address	Essential	test_rate_limit.py
FR-6	A user shall exchange one-to-one text messages in real time	Essential	test_routes.py
FR-7	Text and voice messages shall be end-to-end encrypted with a key the server never holds	Essential	tests/js, test_voice.py

ID	Requirement	Priority	Verified by
FR-8	A user shall record and send a voice note, encrypted as raw bytes	Expected	test_voice.py
FR-9	A user shall share an image, which the system splits into two visual cryptography shares	Essential	test_image_sharing.py
FR-10	Share 2 shall never be written to server storage under any code path	Essential	test_models.py
FR-11	Share 1 shall be retrievable exactly once, through an unguessable capability token	Essential	test_image_sharing.py
FR-12	The system shall verify a stored share against an HMAC-SHA256 tag before release	Essential	test_integrity.py
FR-13	The system shall classify a share as clean or tampered and localise suspected damage	Essential	test_tamper_detector.py
FR-14	The recipient's browser shall run the same classifier independently and agree with the server	Expected	test_onnx_export.py
FR-15	Integrity analysis shall not consume the one-time share access	Expected	test_image_sharing.py
FR-16	A sender shall revoke a message, destroying Share 1 on disk; a recipient's delete shall hide only their own copy	Expected	test_message_lifecycle.py
FR-17	Messages and shares shall expire automatically after one hour	Expected	test_message_lifecycle.py
FR-18	An administrator shall approve, suspend, reinstate and delete accounts strictly below their own rank	Essential	test_admin.py
FR-19	Every administrative action shall be recorded in an audit trail the panel cannot edit	Expected	test_admin.py
FR-20	A user shall change profile, presence and preference settings, and delete their own account after re-authenticating	Expected	test_settings.py
FR-21	The application shall be installable as a Progressive Web App with offline asset caching	Desirable	test_pwa.py
FR-22	An evaluation dashboard shall reproduce the reported measurements without requiring an account	Desirable	test_routes.py

3.4 Non-Functional Requirements

Table 3.2 states the non-functional requirements as measurable targets rather than aspirations, with the measured outcome recorded against each. The measurements themselves are reported in Chapter 5.

Table 3.2: Non-functional requirements, their acceptance criteria and the measured outcome.

ID	Category	Requirement and acceptance criterion	Measured outcome
NFR-1	Correctness	Reconstruction shall be lossless: MSE = 0 and SSIM = 1.0000 on every test image	Met — 6 of 6 images bit-exact
NFR-2	Confidentiality	A single share shall be statistically indistinguishable from uniform random data (entropy ≥ 7.99 bits/px, $\|r\| < 0.01$)	Met — 7.997 bits/px, r = 0.0035
NFR-3	Integrity	Any modification of a stored share shall be detected by at least one layer	Met — 420 of 420 attacks detected
NFR-4	Integrity	False-positive rate of the classifier on clean shares shall be below 2%	Met — 0.71% at threshold 0.50
NFR-5	Performance	A complete integrity check shall take under 150 ms at the 500-pixel size cap	Met — 86 ms median
NFR-6	Performance	Share generation and reconstruction shall be negligible against detection cost	Met — 0.015 ms and 0.003 ms at 256x256

ID	Category	Requirement and acceptance criterion	Measured outcome
NFR-7	Consistency	Server and browser classifiers shall never reach opposite verdicts	Met — 0 disagreements over 200 samples
NFR-8	Security	Passwords shall be hashed with a memory-hard function	Met — Argon2id, 64 MiB, 55 ms
NFR-9	Security	The share integrity key shall be independent of the session signing key	Met — separate key, one-way derived
NFR-10	Accessibility	Text contrast shall meet WCAG 2.1 Level AAA in both themes	Met — 0 failures across 114 elements
NFR-11	Usability	The interface shall be usable at phone widths and honour reduced-motion and reduced-transparency preferences	Met — audited, §5.7
NFR-12	Portability	The system shall run on SQLite in development and PostgreSQL in production from one codebase	Met — asserted in CI
NFR-13	Maintainability	Automated test code shall be comparable in size to application code	Met — 6,307 vs 10,167 lines; 485 tests
NFR-14	Reliability	The full suite shall pass on every change, on two Python versions	Met — CI green

3.5 Design Diagrams

3.5.1 Use case model

Figure 3.3 shows the actors and the use cases available to each. Four actors are distinguished: a visitor who has no account, a registered user, an administrator and a single super-administrator. Each rank inherits the use cases of the rank below it.

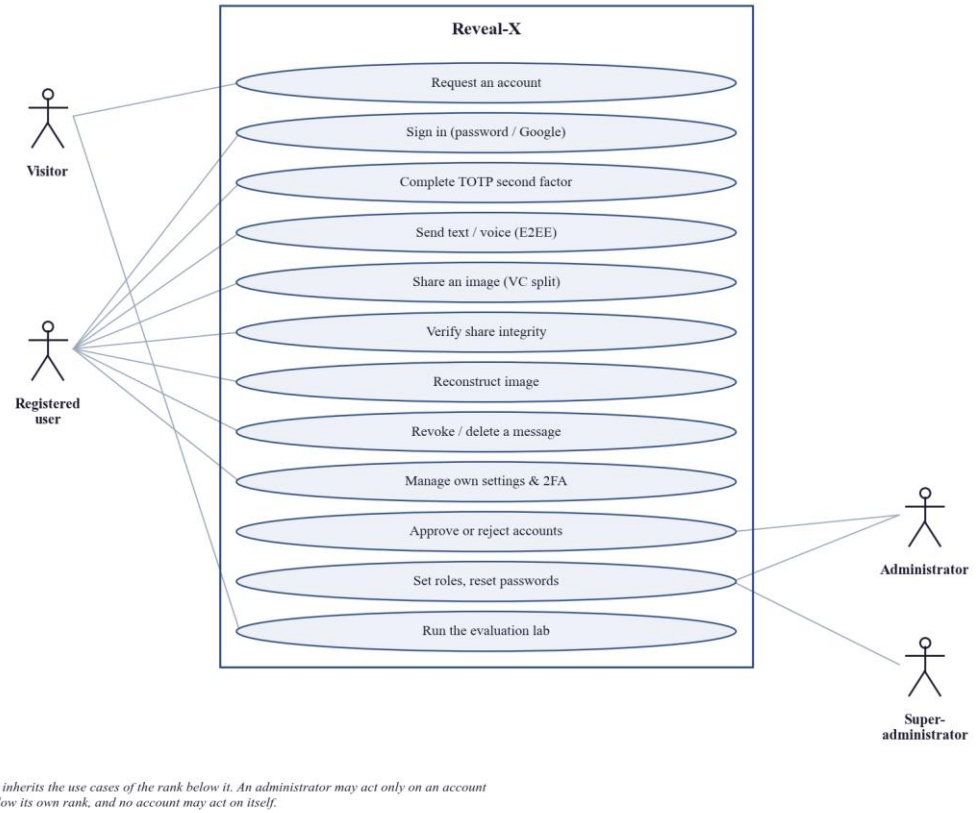


Figure 3.3: Use case model. Ranks inherit downward; an administrator may act only on an account strictly below its own.

Table 3.3 expands the central use case in full. It is given in the detailed format because it is the use case in which the project's distinguishing security properties are exercised.

Table 3.3: Use case specification: Share an image.

Field	Description
Name	Share an image protected by visual cryptography
Actors	Sender (registered user), Recipient (registered user)
Summary	The sender selects an image. The server splits it into two shares, stores Share 1 with an integrity tag, and delivers Share 2 to the recipient over the live socket. The recipient verifies and reconstructs locally.
Pre-conditions	Both parties hold approved, active accounts. The sender has an authenticated socket session. The recipient is online, because Share 2 is never persisted and cannot be delivered later.
Post-conditions	Share 1 and its HMAC tag exist in server storage with a one-hour expiry. Share 2 exists only in the recipient's browser. A capability token has been issued and not yet spent.
Special requirements	Image converted to grayscale; long edge downscaled to at most 500 pixels; PNG only, because a lossy format would break both the HMAC tag and exact reconstruction.
Basic flow	1. The sender selects an image and confirms sending. 2. The system validates the MIME type and size, converts to grayscale and downscales. 3. The system generates Share 1 from the operating system CSPRNG and derives Share 2 by exclusive-or. 4. The system stores Share 1, computes its HMAC-SHA256 tag, and issues a one-time capability token. 5. The system emits Share 2 and the token to the recipient. 6. The recipient requests the integrity analysis, which verifies the tag and runs the classifier without consuming the token.

Field	Description
	7. The recipient fetches Share 1 with the token, which is then spent. 8. The browser reconstructs the image by exclusive-or and displays it.
Alternative flow	4-A. Storage rejects any attempt to persist Share 2; the send fails closed rather than degrading. 6-A. The HMAC tag does not match, or the classifier reports tampering: the recipient is warned and shown the suspect region before deciding whether to open the image. 7-A. The token has already been spent: the request returns 404 and the share is not served a second time. 7-B. The one-hour expiry has elapsed: the message and the share are gone and the request returns 404.

3.5.2 Visual cryptography data flow

Figure 3.4 traces a single image share end to end across the four parties involved. The numbered steps correspond to the basic flow of Table 3.3.

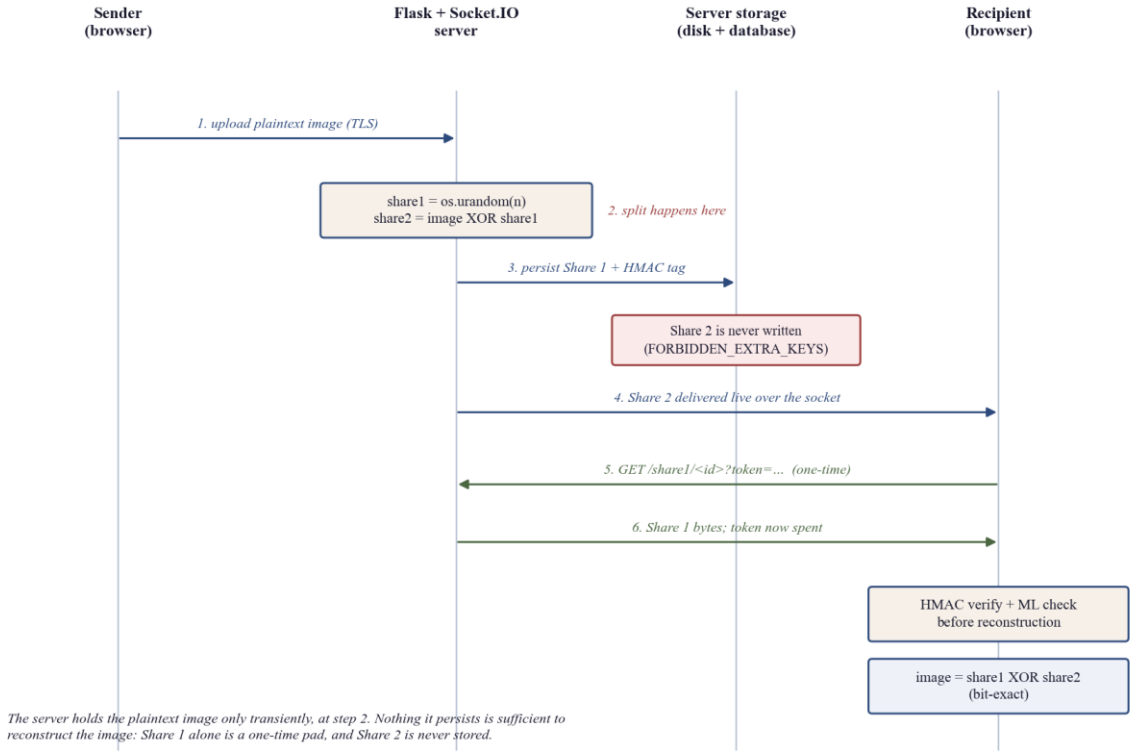


Figure 3.4: Visual cryptography data flow. The server holds the plaintext image only transiently, at step 2.

The property the diagram is drawn to make visible is the asymmetry at steps 3 and 4. Share 1 is persisted and therefore survives; Share 2 is transmitted and then gone. An adversary who obtains the entire server disk obtains only one share, which is a one-time pad and reveals nothing beyond the dimensions of the image.

3.5.3 Database design

The relational schema is deliberately small. Table 3.4 gives the two tables and the columns that carry security meaning. Three indexes support the expiry sweep and the conversation

query. Image content is never stored in the database: Share 1 is a file on disk referenced by the message row, and Share 2 is not stored at all.

Table 3.4: Database schema. Columns marked with an asterisk carry direct security significance.

Table	Column	Type	Purpose
users	id	TEXT PK	Opaque identifier
users	username	TEXT UNIQUE	Display name; escaped on all output
users	password_hash *	TEXT	Argon2id hash; empty for Firebase-only accounts
users	firebase_uid *	TEXT	Account key for Google sign-in; never the email address
users	totp_secret *	TEXT	Shared secret for the second factor
users	role *	TEXT	user | admin | superadmin
users	account_state *	TEXT	pending | active | suspended
users	public_key	TEXT	ECDH P-256 public half, SPKI base64
users	email, bio, status, profile_image, last_seen, auth_provider, created_at	TEXT	Profile and presence
messages	id	TEXT PK	Opaque identifier
messages	sender_id, recipient_id	TEXT	Participants; every read is filtered on these
messages	type	TEXT	text | voice | image
messages	content	TEXT	Ciphertext for text and voice
messages	extra *	TEXT (JSON)	Share 1 reference, HMAC tag, capability token; Share 2 keys are refused here
messages	share1_accessed *	BOOLEAN	Enforces one-time access
messages	expires_at *	TEXT	One-hour time-to-live
messages	deleted_by	TEXT (JSON)	Per-participant deletion scope
messages	read_at, created_at	TEXT	Receipts and ordering

3.6 Hardware and Software Requirements

Table 3.5 states the development and deployment requirements. The system is deliberately modest in its demands: there is no GPU requirement and no deep-learning framework anywhere in the dependency tree, because the detector is a 45-tree random forest of 107.7 KB that exports to a 40.2 KB ONNX file.

Table 3.5: Hardware and software requirements.

Category	Requirement
Server hardware (development)	Any x86-64 or ARM64 machine, 4 GB RAM, 2 GB free disk. No GPU.
Server hardware (deployment)	1 vCPU and 1 GB RAM is sufficient for the measured load; shares are incompressible, so disk should be sized at the uncompressed image volume per hour.
Client hardware	Any device able to run a current browser; the in-browser classifier runs in 0.0125 ms per sample.
Operating system	Windows 10/11, Linux or macOS for development; Linux for deployment.
Runtime	Python 3.10 or later; CI exercises two versions.
Server libraries	Flask 3.1.3, flask-socketio 5.7.0, python-socketio 5.16.2, gunicorn 23.0.0, argon2-cffi 23.1.0, cryptography 50.0.0, numpy, opencv-python-headless, Pillow, scikit-learn, joblib, psycopg2-binary, firebase-admin, pyotp, qrcode.

Category	Requirement
Database	SQLite 3 in development; PostgreSQL 13 or later in production.
Browser	Chrome 90+, Edge 90+, Firefox 88+ or Safari 15+. A secure context (HTTPS or localhost) is required: without it the Web Crypto API, the service worker and microphone access are all unavailable.
Client libraries	onnxruntime-web, vendored locally rather than loaded from a CDN. No framework and no build step.
Development tooling	pytest, Node.js 18+ for the browser test runner, pip-audit for dependency review.

3.7 Threat Scenarios

The assets this system protects, in order of value, are: the content of shared images, the content of text and voice messages, user credentials, the share capability tokens, and the keys behind the integrity tags and the session cookie. Five concrete scenarios drove the design.

- **Scenario 1 — the server is compromised or compelled.** An adversary obtains the database and the disk. For images the system must ensure the stolen material is insufficient to reconstruct anything; for text and voice it must ensure the stored content is ciphertext the server cannot decrypt.
- **Scenario 2 — a stored share is edited in place.** An adversary with write access to share storage modifies Share 1 so that the recipient reconstructs an altered image. The system must detect the modification before reconstruction and indicate where the damage lies.
- **Scenario 3 — an authenticated user reaches beyond their own data.** A signed-in account attempts to read, revoke or analyse a message it is not party to, or to fetch a share using a guessed identifier.
- **Scenario 4 — credential attack.** An adversary runs a password list against one account, sprays one password across many accounts, or, holding a correct password, brute-forces the six-digit second factor.
- **Scenario 5 — resource abuse by an unauthenticated client.** An anonymous client drives expensive work — the laboratory pipeline, or the memory-hard password hash behind account registration — fast enough to deny service to legitimate users.

3.8 Threat Modeling Techniques

The threat model was developed using STRIDE, chosen because it enumerates threat categories against each element of a data-flow diagram, and Section 3.5.2 already expresses the system as one. STRIDE suits a system whose assets are well understood and

whose question is coverage rather than discovery. Attack trees were considered and used informally for the second-factor analysis in Section 5.1, where the quantity of interest is the expected number of guesses rather than a category of threat. DREAD was not used for scoring, because its ratings are subjective and the project had access to direct measurement for most of the risks in question.

Table 3.6 applies STRIDE to the elements of the data flow. Each row names the element, the threat, and the control that answers it; controls shown in the final column are verified in Chapter 5.

Table 3.6: STRIDE analysis against the elements of the data-flow diagram.

Category	Element	Threat	Control
Spoofing	Client → server socket	An attacker presents as another user	Argon2id credential check; optional TOTP; server-verified Firebase ID tokens with issuer, audience, expiry and revocation checks; accounts keyed on Firebase UID, never email
Spoofing	Public key exchange	A relay substitutes its own ECDH key	Short public-key fingerprint displayed in the conversation header for out-of-band comparison
Tampering	Stored Share 1	Bytes on disk are modified	HMAC-SHA256 over the written bytes, compared in constant time; 13-feature classifier localising damage to 16x16 blocks
Tampering	Message in flight	Ciphertext altered in transit	AES-256-GCM is authenticated encryption; TLS 1.3 with HSTS beneath it
Repudiation	Administrative actions	An administrator denies deleting an account	Audit trail written to the log rather than to the database, so it cannot be edited through the same panel
Information disclosure	Server storage	Database and disk are stolen	Share 1 alone is a one-time pad; Share 2 is never persisted; text and voice are stored as ciphertext
Information disclosure	Share endpoint	A share is fetched repeatedly or by a third party	secrets.token_urlsafe(32) capability token; one-time access enforced by share1_accessed; every read filtered on sender or recipient
Information disclosure	Error messages	A refusal reveals why an account matters	Deletion refusals cite the password only, never administrator status
Denial of service	Registration	Anonymous requests drive a 64 MiB memory-hard hash	Five requests per address per ten minutes, checked before the hash is computed
Denial of service	Laboratory endpoints	Unauthenticated clients run the full pipeline	Twenty requests per address per minute; partial, and recorded as a limitation in §6.3
Elevation of privilege	Administration panel	A user escalates to administrator, or an administrator acts on a peer	Three ranks; an account may act only on one strictly below its own and never on itself; the single superadmin comes from the environment at startup and cannot be created through the interface
Elevation of privilege	Second factor	A password holder brute-forces the six-digit code	Wrong codes recorded against the account and the address; the throttle is consulted before each guess and is not cleared by a correct password

3.9 Threat Resistance Model

Table 3.7 states the resulting position against each adversary the system was designed for. A threat model that lists only successes is marketing, so the rows that are not defended are given the same prominence as the rest. Chapter 5 reports the measurements behind the "Defended" entries.

Table 3.7: Threat resistance model. Three adversaries are not defended against, and are stated as plainly as the rest.

Adversary	Capability	Outcome	Control or reason
Passive network observer	Reads traffic in transit	Defended	TLS 1.3 beneath AES-GCM end-to-end encryption
Active network attacker	Modifies traffic in flight	Defended	GCM is authenticated encryption; HSTS; same-origin socket policy
Another signed-in user	Holds a valid account	Defended	Every read filtered on sender or recipient; one-time capability tokens
Unauthenticated attacker	Can reach the server over HTTP	Partly defended	Login throttling and a registration cap; the two laboratory endpoints are rate limited but not authenticated
Holder of a stored Share 1	Obtains a backup or the disk	Defended	Share 1 alone is a one-time pad; Share 2 is never persisted
Someone who edits a stored share	Write access to share storage	Defended	HMAC detected every attack tested; the classifier localises six of seven classes
Password holder attacking the second factor	Knows the password	Defended	Durable per-account throttle on wrong codes; roughly 333,000 expected guesses against a capped rate
Curious server operator	Full server access	Split	Text and voice are unreadable; images are visible at split time
Compromised endpoint	Malware in a participant's browser	Not defended	Plaintext exists before encryption and after decryption
Attacker with a past private key	Captured traffic plus a later key compromise	Not defended	No forward secrecy; a ratchet [19] would be required
Adaptive adversary against the classifier	Knows the 13 features and crafts an attack preserving them	Not defended by the classifier	Demonstrated by the noise attack in §5.6.4; the HMAC layer catches it in 60 of 60 trials

3.10 Chapter Summary

This chapter specified twenty-two functional and fourteen non-functional requirements, each with a verification route, and presented the architecture at both the layered and the component level. The design diagrams give the use case model, a full specification of the central use case, the corrected visual cryptography data flow, and the database schema. The hardware and software requirements record a system that needs no GPU and no deep-learning framework.

The threat model was developed with STRIDE over the elements of that data flow, producing twelve threat-control pairs, and consolidated into the resistance model of Table 3.7. Three adversaries are explicitly not defended against: a compromised endpoint, an attacker holding a past private key, and an adaptive adversary who crafts an attack to preserve the classifier's features. The last of these is not hypothetical, and Chapter 5 measures it. Chapter 4 sets out the proposed solution and the methodology by which these requirements were met and evaluated.

Chapter 4:

Proposed Solution

Chapter 4: Proposed Solution

4.1 Introduction

This chapter sets out the solution proposed against the problem statement of Section 2.5 and the methodology by which it is evaluated. It describes the visual cryptography model and the two-layer integrity design, states how the evaluation data was generated and prepared, names the tools and techniques used, and defines the metrics on which Chapter 5 reports.

The approach is quantitative and experimental. Every claim in Chapter 5 is produced by a script that can be re-run against the repository; none is quoted from the literature or from memory. Where a measurement contradicts something asserted earlier in the project, the contradiction is reported rather than reconciled, and Section 5.6.1 contains one such case.

4.2 Proposed Model

4.2.1 The visual cryptography scheme

Reveal-X implements a (2,2) XOR-based visual cryptography scheme on grayscale images. It is the two-party case of threshold secret sharing [2], [3] specialised to images, in the computed-exclusive-or branch described by Tuyls et al. [5]. For an original image O of height h and width w the construction is:

```
share1   = os.urandom(O.size)      # cryptographically secure mask
share2   = O XOR share1            # the complement of the mask
recovered = share1 XOR share2 = O  # exact, by associativity
```

Two choices in that construction carry the security argument. The first is the source of randomness. Share 1 is drawn from the operating system CSPRNG rather than from the NumPy default generator; a share produced from a seeded pseudo-random stream is reproducible by anyone who recovers the seed, which would reduce the scheme from encryption to obfuscation. The second is that the mask is drawn afresh for every message and never cached. With a uniform random Share 1, each Share 2 pixel is a one-time pad of the corresponding original pixel, which is precisely the construction Shannon proved to be perfectly secret [9]; reusing a pad would be catastrophic for the same reason it is catastrophic for any one-time pad.

Classical visual cryptography [1] stacks printed transparencies and decodes with the human visual system, paying for that convenience with pixel expansion of two to four

times and a halving of contrast. Because decoding here is performed in software, those costs would buy nothing. Choosing XOR converts a lossy scheme with storage expansion into a lossless one with none.

One property of the scheme is recorded now as a limitation rather than discovered later. A modern block cipher exhibits the avalanche effect, in which changing one plaintext bit changes about half the ciphertext bits, measured as an NPCR of roughly 99.609%. The XOR split has no such diffusion: altering a single pixel of the original and re-splitting with the same Share 1 changes exactly one pixel of Share 2, an NPCR of 0.001526%. This is not a defect, because confidentiality rests on the pad being uniform and used once rather than on diffusion, but it does mean that an adversary holding Share 2 who can observe how it changes across retransmissions of a near-identical image learns which positions changed.

4.2.2 Two-layer integrity

A keyed message authentication code alone would detect any modification to a stored share, so the decision to add a classifier needs justification. An HMAC [11] is a binary oracle over the whole file: it reports that something changed, never what or where, and it is useless once a share has been legitimately re-encoded, because a lossy round trip breaks the tag just as thoroughly as an attack does. A statistical classifier is approximate, but it localises damage to a region, produces a calibrated confidence, survives transformations that preserve content, and can explain its verdict in terms a user can read.

The proposed model therefore attaches both to every stored share and runs both before reconstruction is permitted. The hypothesis under test, evaluated in Section 5.6.5, is that the two fail in different places and that neither is redundant. The classifier additionally runs a second time inside the recipient's browser, because one-time access means that after the recipient fetches the share the browser is the only party that can still say anything about it — and asking the server whether the file it supplied is intact asks the party that stored it.

4.2.3 The detector

The detector does not operate on raw pixels. It extracts thirteen statistical features, chosen because a valid Share 1 has a known expected profile: a uniform histogram, zero adjacent-pixel correlation, maximum entropy and a mean near 127.5. Every feature measures a departure from that profile. The approach follows the rich-model tradition in image

forensics [27] at a much smaller scale, appropriate to a vector that must also be computed in a browser.

Table 4.1: The thirteen-feature vector, its interpretation, and the tampering signal each feature is intended to capture.

#	Feature	What it measures	Why a tampered share moves it
1	mean	Average grey level	Brightness shifts and block fills pull the mean away from 127.5
2	std	Standard deviation of grey levels	Flat regions reduce spread; clipping increases it
3	entropy	Shannon entropy over the 256-bin histogram	Any inserted structure costs entropy; a clean share sits at 8.000
4	hist_chi2	Chi-square distance of the histogram from uniform	Edits concentrate mass on particular grey levels
5	hist_l1_uniform	L1 distance of the histogram from uniform	A robust companion to the chi-square term, less dominated by outliers
6	corr_h	Correlation of horizontally adjacent pixels	Blur and resampling introduce neighbour correlation where there was none
7	corr_v	Correlation of vertically adjacent pixels	As above, in the orthogonal direction
8	laplacian_var	Variance of the Laplacian response	A smoothness proxy: blurring lowers it, scratches raise it locally
9	block_mean_std	Spread of 16x16 block means	A uniform share has near-identical block means; a patch breaks that
10	block_std_mean	Mean of 16x16 block standard deviations	Detects locally flattened regions that global statistics hide
11	unique_ratio	Distinct grey levels present, over 256	Block replacement and quantisation reduce the palette
12	zero_ratio	Fraction of pure black pixels	Block fills and clipping create runs of exact zeros
13	sat_ratio	Fraction of pure white pixels	The saturated counterpart of the previous feature

Three constraints shaped the set. It must be cheap enough to run on every share before reconstruction; it must be reproducible bit-for-bit in JavaScript so that the same model can run in the browser; and it must be interpretable, so the application can tell a user why a share was flagged rather than only that it was.

The shipped model is a RandomForest [21] of 45 trees with a maximum depth of 10, class-weighted to balance the two classes, holding 1,109 decision nodes at a mean realised depth of 7.7. It occupies 107.7 KB as a joblib file on the server and 40.2 KB once exported to ONNX for the browser. Section 5.6.6 reports the comparison against seven alternative learners that justifies this choice.

4.3 Data Collection

No external image corpus is used. Data is generated at run time by the project's own attack simulator, for a reason that is specific to this problem: the clean class is not a natural image but a uniform random share, which can be synthesised exactly, and the tampered

class is a clean share with a known, parameterised modification applied. Generating both removes any confound between the two classes other than the attack itself.

Seven attack types were chosen to span the plausible ways a stored share is damaged or manipulated, from deliberate editing to incidental format loss. They are defined in Table 4.2.

Table 4.2: The seven simulated attacks used for training and evaluation, with the real-world case each stands in for.

Attack	Operation	Represents
block	Replace a rectangular region with a constant grey level	Deliberate censorship or region substitution
noise	Replace a fraction of pixels with fresh random values	Bit-level corruption, or an attacker mimicking share statistics
blur	Gaussian blur applied to a sub-region	Smoothing to erase detail; resampling artefacts
brightness	Add a constant offset to a sub-region, then clip	Exposure or level manipulation
crop	Crop the border away and rescale to the original size	Reframing, and the resampling it forces
scratch	Draw several thick random lines	Overwriting or defacing part of the share
jpeg	Encode and decode as JPEG at a reduced quality factor	Lossy re-encoding by a well-meaning intermediary

Attack strength is sampled uniformly between 0.18 and 0.55 so that the classifier sees a range of severities rather than one convenient setting. Three separate datasets are used, and keeping them apart is what makes the results in Chapter 5 meaningful: reporting a model's accuracy on the data it was fitted to is the most common way an evaluation becomes worthless.

Table 4.3: The three datasets. The evaluation set is fourteen times the size of the internal test split and was never seen during fitting.

Dataset	Seed	Samples	Composition	Used for
D_train	42	240	120 clean + 120 tampered, paired	Fitting the shipped model (75% split) and every comparison model
D_eval	2026	840	420 clean + 420 tampered (60 per attack)	Every accuracy, ROC and per-attack figure in Chapter 5
D_noise	7331	400	40 samples at each of 10 noise strengths	The blind-spot analysis in Section 5.6.4

Two details of the construction matter. Every tampered sample is derived from the clean sample beside it in the dataset, so the only difference between a negative and a positive example is the attack itself; share-to-share variation is removed from the comparison by construction. And each share is 96 by 96 pixels, the scale at which the model was trained to operate.

4.4 Data Pre-Processing

Pre-processing happens at two points, and they serve different purposes.

In the application, an uploaded image is validated against an allowed set of MIME types, converted to a single grayscale channel, and downscaled so that its long edge is at most 500 pixels. The grayscale conversion is required because the scheme operates on one channel; the downscale bounds both the storage cost, which Section 5.6.2 shows to be the full uncompressed size of the image, and the detection latency, which Section 5.6.9 shows to scale roughly linearly in pixel count. Shares are handled as PNG throughout and never as JPEG, because lossy re-encoding would break both the integrity tag and exact reconstruction.

In the detector, pre-processing is deliberately minimal. No normalisation, whitening or augmentation is applied to the pixel array, because the features are themselves summary statistics and are expressed as ratios and correlations rather than absolute counts. Feature values are computed directly from the raw 8-bit array, which is what allows the JavaScript implementation to reproduce them to within a relative tolerance of 1e-12 and keeps the browser and server verdicts in agreement.

One pre-processing case is refused rather than approximated. A colour image decoded through a different path in the browser would produce a different grayscale conversion and therefore a different feature vector, so the browser raises an error instead of returning a plausible number. A second opinion that is quietly wrong is worse than no second opinion.

4.5 Tools and Techniques

The project uses established libraries rather than bespoke implementations wherever a cryptographic or numerical primitive is involved, on the principle that a final-year project should not be inventing its own cryptography. Table 4.4 records what is used and why.

Table 4.4: Tools and techniques, with the role each plays.

Area	Tool or technique	Role
Visual cryptography	NumPy, os.urandom	Share generation and exclusive-or reconstruction
Integrity	HMAC-SHA256 [11], constant-time comparison	Keyed tag over the bytes as written to disk
Machine learning	scikit-learn [22] RandomForest [21]	Tamper classification over the thirteen features
In-browser inference	ONNX Runtime Web [34]	The same model, executed in the recipient's browser
Imaging	OpenCV [24], Pillow	Grayscale conversion, downscaling, blur and JPEG attack simulation
Key agreement	ECDH on NIST P-256 [10], [15]	Per-conversation shared secret
Key derivation	HKDF-SHA256 [12]	Extract-and-expand; a raw ECDH output is not a

Area	Tool or technique	Role
		uniform key
Content encryption	AES-256-GCM [13], [14]	Authenticated encryption of text and voice
Password hashing	Argon2id [17]	Memory-hard, GPU-resistant credential storage
Second factor	TOTP, RFC 6238 [16]	Time-based one-time passwords
Transport	TLS 1.3 [18] with HSTS	Defence in depth beneath E2EE
Web framework	Flask, Flask-SocketIO	HTTP routing and real-time events
Evaluation	Chi-square, Shannon entropy [8], adjacent-pixel correlation, PSNR, SSIM [25], ROC and AP [26]	The metrics defined in Section 4.6
Statistical testing	Mann-Whitney U, Cohen's d	Model-independent confirmation of feature separation
Verification	pytest, Node.js test runner, mutation testing	Automated tests, and confirming those tests can fail

4.6 Evaluation Metrics

Table 4.5 defines the metrics used in Chapter 5 and states the criterion by which each is judged. Separating the definition from the result is deliberate: the thresholds were fixed before the measurements were taken.

Table 4.5: Evaluation metrics and their acceptance criteria.

Metric	Definition	Criterion
MSE	Mean squared error between original and reconstruction	Exactly 0
PSNR	Peak signal-to-noise ratio in decibels	Unbounded, given MSE = 0
SSIM [25]	Structural similarity across luminance, contrast and structure	Exactly 1.0000
Shannon entropy [8]	Bits per pixel over the 256-bin histogram	≥ 7.99 for a share (maximum 8.000)
Adjacent-pixel correlation	Pearson r between horizontally and vertically neighbouring pixels	$\|r\| < 0.01$ for a share
Chi-square p-value	Goodness of fit of the histogram to uniform	Fails to reject uniformity for a share
NPCR	Proportion of pixels changed by a one-pixel input change	Reported, not targeted; see §4.2.1
Accuracy	(TP + TN) / n on the independent set	> 90%
Precision	TP / (TP + FP)	> 95%; a false alarm on a clean share costs user trust
Recall	TP / (TP + FN)	Reported with its ceiling explained
F1 score	Harmonic mean of precision and recall	Reported
False positive rate	FP / (FP + TN)	< 2%
ROC AUC and average precision [26]	Threshold-independent separability	Reported
Cohen's d; Mann-Whitney U	Effect size and significance of each feature, computed independently of the model	Reported for the leading features
Latency	Median wall-clock time per stage, by image size	< 150 ms for a full check at the 500-pixel cap
Verdict agreement	Server versus browser over identical inputs	Zero disagreements

4.7 Chapter Summary

This chapter has set out the proposed solution: a (2,2) XOR visual cryptography scheme with a cryptographically seeded, never-reused mask, protected by two independent integrity layers whose complementarity is stated as a testable hypothesis rather than assumed. It has described the thirteen-feature detector and the reasoning behind each feature, the synthetic data-collection procedure and the three separated datasets, the minimal pre-processing that keeps the browser and server implementations in agreement, the tools used, and the metrics with their acceptance criteria fixed in advance. Chapter 5 reports the implementation and the measurements.

Chapter 5:

Implementation and Testing

Chapter 5: Implementation and Testing

5.1 Security Properties Testing

The security properties were tested as properties, by asserting them in automated tests that run on every change, rather than by inspection. Five are singled out here because they are the properties the design rests on.

- **Share 2 is never persisted.** The storage layer holds a constant, FORBIDDEN_EXTRA_KEYS, naming the keys under which Share 2 could be written, and refuses any write containing them. The test suite asserts the refusal directly, so a future change that began persisting Share 2 would fail the build rather than silently weakening the central guarantee.
- **One share reveals nothing.** Tests assert that neither share correlates with the original and that the sender never receives Share 2 or the capability token. The statistical evidence is in Section 5.6.2.
- **Share access is single-use.** The capability token is drawn from secrets.token_urlsafe(32), and a second request for the same share returns 404. Integrity analysis deliberately does not consume the access, so a recipient can be warned before deciding whether to open the image.
- **Credential attacks are bounded.** Failed sign-ins are throttled on two buckets: eight attempts per account and fifty per network address within a fifteen-minute window. The per-address limit is deliberately loose, because everyone behind one router shares an address and a tight value would punish bystanders.
- **The second factor is not a formality.** The attempt limit was originally counted against the socket identifier, so disconnecting bought five fresh guesses indefinitely, and a correct password cleared the throttle before the code was requested. Six digits with a one-step drift tolerance leaves roughly three valid codes at any instant, so about 333,000 expected guesses — at the rate the socket allowed, a few hours for anyone already holding the password. Wrong codes are now recorded durably against both buckets and the throttle is consulted before each guess.

Cross-site scripting prevention is tested against the OWASP evasion catalogue [35], [41] rather than against expectations the project set itself, exercising the escaping path on the shipped client code. Password storage uses Argon2id [17] at a measured 55 ms and 64

MiB per hash; that cost is the point of the algorithm, and it is also why unauthenticated access to it had to be capped, as described in Section 5.4.

One further property was added during hardening and is worth recording because it is invisible from the outside. The key behind the share integrity tag was originally the session signing key. One secret doing two unrelated jobs means that leaking it grants both the ability to forge a session and the ability to re-tag an altered share so that it verifies clean (CWE-323). The tag now uses a separate key, derived one-way from the session key when not configured explicitly, so existing deployments continue to work without a configuration change.

5.2 System Setup

The server runs on Python 3.10 or later behind gunicorn in production and the Flask development server locally. Configuration is supplied entirely through environment variables, with no secrets in source control. Table 5.1 lists those that affect security behaviour.

Table 5.1: Security-relevant configuration.

Variable	Effect if unset	Recommended
REVEAL_X_SECRET_KEY	Random per run, so sessions and derived keys do not survive a restart	Set explicitly; see the note below
REVEALX_SHARE_HMAC_KEY	Derived one-way from the session key	Set explicitly in production
REVEALX_ADMIN_USER / _PASSWORD	No super-administrator is created	Set at first start
REVEALX_TRUST_PROXY	Throttling keys on the proxy address, placing every user in one bucket	Set only behind a proxy you control
FLASK_DEBUG	No auto-reload	Leave unset in production

One operational consequence deserves emphasis because it produces a failure that looks like a security alert. If the secret key is left unset it is random for each run, which makes the derived share integrity key random too. A share tagged before a restart then fails verification afterwards and is reported as tampered — a false alarm on the headline feature, indistinguishable on screen from a genuine detection.

The cryptographic parameters of the end-to-end encrypted channel are given in Table 5.2, and the handshake is shown in Figure 5.1.

Table 5.2: Cryptographic parameters of the end-to-end encrypted channel.

Property	Choice	Rationale and reference
Key agreement	ECDH on NIST P-256	Diffie-Hellman over elliptic curves [10]; parameters per NIST SP

Property	Choice	Rationale and reference
		800-56A [15]
Key derivation	HKDF-SHA256	Extract-and-expand, RFC 5869 [12]; a raw ECDH output is not a uniform key
Content encryption	AES-256-GCM	FIPS 197 [13] with GCM per NIST SP 800-38D [14]; authenticated encryption
Nonce	96-bit, fresh per message	GCM nonce reuse under one key is catastrophic, so it is never reused
Private key storage	Non-extractable CryptoKey in IndexedDB	Web Cryptography API [33]; the key cannot be read out by script, including ours
Public key transport	Base64 SPKI over the socket	The server relays public halves; it never sees a private half
Password hashing	Argon2id	Memory-hard, resistant to GPU attack [17]
Second factor	TOTP, RFC 6238	Time-based one-time passwords [16]
Transport	TLS 1.3 with HSTS	RFC 8446 [18]; defence in depth beneath the E2EE layer

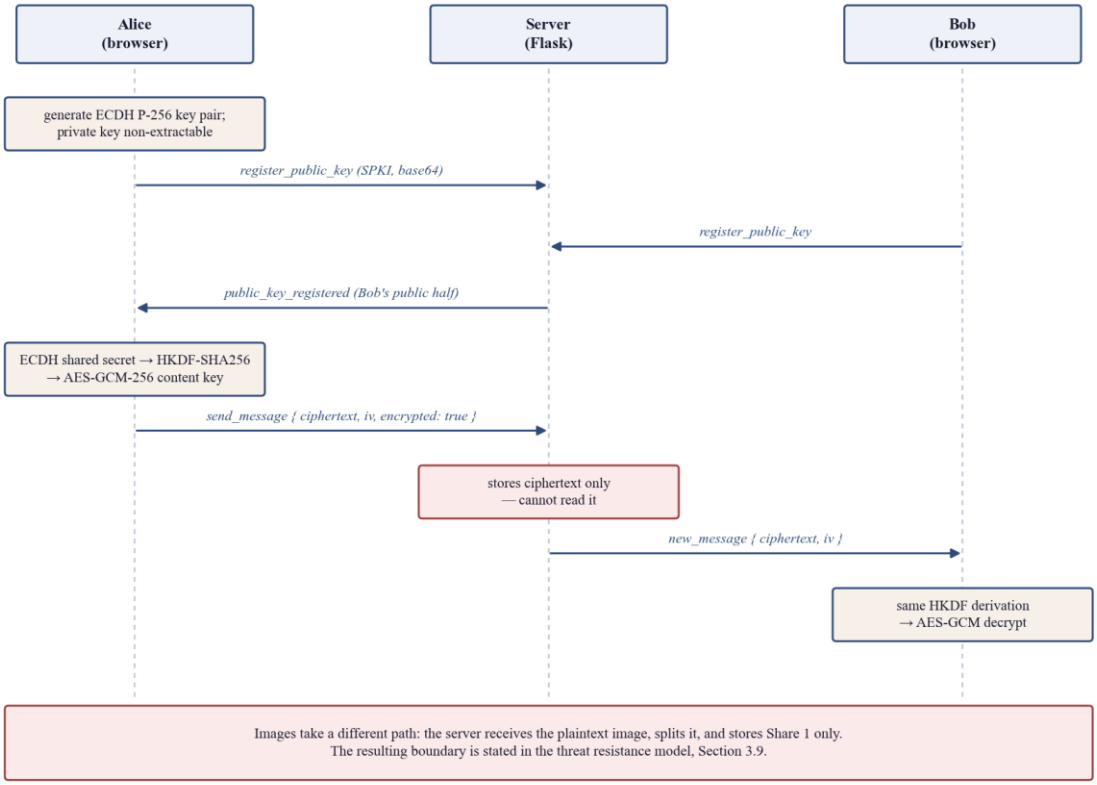


Figure 5.1: The end-to-end encryption handshake and message path, including the image exception.

A short fingerprint of each public key is displayed in the conversation header so that two users can compare it out of band, which is the standard defence against a relay substituting its own key. Text and voice travel this path. Images do not, and the boundary is stated here because it is easy to misread: the browser uploads the image and the server performs the split, so the plaintext image is in server memory at that moment. The guarantee for images is different and narrower — nothing the server persists is sufficient to reconstruct the image.

5.3 System Integration

The application integrates three subsystems behind one interface: the real-time messaging layer, the visual cryptography and integrity core, and the authentication stack. Integration is through nine HTTP routes and thirty-two Socket.IO events.

Messaging is duplex over Socket.IO, carrying message delivery, typing indicators, read receipts, presence, reactions, forwarding, replies, voice notes and revocation. The image path crosses all three subsystems: an authenticated socket event accepts the upload, the visual cryptography core produces the shares, the integrity layer tags Share 1 and issues a capability token, and the socket layer delivers Share 2 to the recipient while the token is spent over HTTP.

Persistence runs on SQLite in development and PostgreSQL in production from a single codebase, with portability asserted in continuous integration rather than assumed; a dedicated test module checks that the SQL used is accepted by both engines. The browser integrates ONNX Runtime Web, vendored locally rather than loaded from a content delivery network, so that the in-browser classifier has no third-party runtime dependency. The application is an installable Progressive Web App whose service worker caches static assets with a versioned cache name.

Continuous integration runs both suites on every change across two Python versions, builds the container, and boots the production server from the Procfile to assert that it serves, completes a Socket.IO handshake and reports an unconfigured Firebase correctly — because gunicorn does not run on Windows, and an unverified deployment command is only a note.

5.4 Penetration Testing and Vulnerability Assessment

The template marks this section optional. It is included because the project is a security project and an assessment was in fact carried out, by the project team rather than independently; that limitation is recorded in Section 6.3.

The review followed the OWASP Top 10 structure [35] over the application’s entry points: the HTTP routes, the socket events, the uploaded image and audio paths, and the administrative panel. Eight findings were produced. Three were judged exploitable enough to fix immediately and are described below; the remainder were recorded as hardening items, of which the outstanding ones appear in Section 6.3.

- **Second-factor brute force (A07).** Described in Section 5.1. The attempt counter was bound to the socket rather than to the account. Fixed, and the fix verified by a test that opens its sockets in advance.
- **Unauthenticated expensive work (A06).** Account registration performed an Argon2id hash — 64 MiB and 55 ms — before any rate check, on a single-worker deployment. Requests are now capped at five per address per ten minutes, and the cap is evaluated before the hash rather than after it.
- **Key reuse across purposes (A02, CWE-323).** The share integrity tag was keyed with the session signing key. It now has its own key.
- **Secrets collected through a browser dialog (A07).** Four flows — the administrator password reset, turning two-factor off, and setting or clearing the device passcode — used window.prompt(), which renders the typed value in cleartext and cannot mask it. They now use a masked in-app dialog that also validates without discarding the entry. A rule test fails the build if prompt() reappears anywhere in the shipped scripts.

Every dependency is now pinned exactly, with a `requirements.lock` recording the full resolved tree, because the floor pins had drifted a long way from anything the suite had been run against: `numpy>=1.26` was resolving to 2.4 and `opencv-python-headless>=4.8` had crossed a major version to 5.0. A fresh install would not have reproduced the environment these results were measured in, and the tamper detector is a joblib file, which is sensitive to the scikit-learn version it was fitted under. Dependency review was performed with pip-audit. Two pinned versions in the requirements file carry comments recording the advisories that forced them: gunicorn 23.0.0 replaces a version carrying two request-smuggling advisories, and cryptography 50.0.0 is the first release clear of seven advisories affecting the previous pin. The audit reports no known vulnerabilities against the current manifest.

Each of the three fixes was mutation-tested: the fix was deliberately reverted to confirm that the new test fails. That exercise found one test passing vacuously — a sequential reconnect loop was being stopped by the login gate and never reached the check it claimed to cover. Rewritten to open its sockets in advance, it showed the unfixed code permitting thirty guesses against a ceiling of thirteen. A test that cannot fail proves nothing, and only mutation revealed it.

5.5 Test Cases

The automated suite comprises 416 server-side tests across nineteen modules and 69 browser-side tests across five, totalling 485. Table 5.3 gives the distribution and what each module holds in place.

Table 5.3: The automated test suite by module. The distribution reflects where a defect would be most costly.

Test module	Tests	What it holds in place
test_admin.py	47	Role hierarchy; refusal from non-admins and anonymous sockets
test_auth.py	45	Argon2id hashing, TOTP, Firebase token verification, clock skew, throttle durability
test_models.py	40	Storage layer, including the refusal to persist Share 2
test_pwa.py	38	Manifest, service worker scope, offline cache behaviour
test_message_lifecycle.py	27	Delivery, deletion scope, expiry, re-authentication before account deletion
test_settings.py	26	Profile and preference updates, input validation
test_rate_limit.py	25	Login throttling on both buckets, and what does not count
test_voice.py	24	Voice message encryption and playback path
test_image_sharing.py	22	Share generation, one-time capability tokens, revocation
test_deployment.py	20	Procfile, Dockerfile and server configuration consistency
test_tamper_detector.py	15	Feature extraction, prediction, heatmap, statistical fallback
test_routes.py	15	HTTP routes and socket event contracts
test_glass_fallbacks.py	13	Reduced-motion and reduced-transparency fallbacks per page bundle
test_mobile_layout.py	12	Control sizing and alignment at phone widths
test_onnx_export.py	10	Server-to-browser model agreement
test_integrity.py	7	HMAC generation, verification, constant-time comparison
test_metrics.py	6	MSE, PSNR and SSIM implementations
test_vc_core.py	5	Share generation and bit-exact reconstruction
test_secret_prompts.py	19	That no shipped script collects a secret through a browser dialog
tests/js (5 modules, node --test)	69	E2EE primitives, feature parity, service worker, HTML sanitisation, the masked secret dialog
Total	485	416 server-side, 69 browser-side

Four test cases are specified in full below, chosen because each exercises one of the properties the design depends on. The remainder follow the same structure in the repository.

Table 5.4: Test case TC-01: Share 2 is never written to server storage.

Test Case ID	TC-01	Test Date	November 2026
Test case version	2.0	Use case reference(s)	FR-10, NFR-2
Revision history	Supersedes the FYP Part 1 manual check	Verified by	test_models.py
Objective	Confirm that no code path can persist Share 2, so that a full disk compromise yields only a one-time pad.		
Product / module	Storage layer; visual cryptography core.		
Environment	pytest against a temporary database; no		

	external services.		
Assumptions	The attack surface is the extra JSON column on the message row, which is the only place Share 2 could plausibly be written.		
Pre-requisite	A sender and a recipient account exist and an image has been accepted.		
Step	Execution description	Expected result	
1	Send an image through the socket handler	Shares are generated; Share 1 is written with its HMAC tag	
2	Inspect the stored message row and the share directory	No value under any forbidden key is present	
3	Attempt a direct write containing a forbidden key	The storage layer refuses the write and raises	
Result	Passed		

Table 5.5: Test case TC-02: A tampered share is detected and localised before reconstruction.

Test Case ID	TC-02	Test Date	November 2026
Test case version	2.0	Use case reference(s)	FR-12, FR-13, NFR-3
Revision history	Supersedes the FYP Part 1 manual check	Verified by	test_tamper_detector.py, test_integrity.py
Objective	Confirm that a modified stored share is detected by at least one layer and that the damaged region is identified.		
Product / module	Integrity layer; tamper detector.		
Environment	pytest with the shipped model file present.		
Assumptions	The block attack is used because it is representative of deliberate region substitution.		
Pre-requisite	A clean share exists with a valid HMAC tag.		
Step	Execution description	Expected result	
1	Record the classifier score for the clean share	P(tampered) is low; measured 0.0111	
2	Apply a block attack to the stored bytes	The file differs from the tagged bytes	
3	Verify the HMAC tag	Verification fails	
4	Re-score with the classifier and request the heatmap	P(tampered) = 1.0000 and the heatmap localises the correct 16x16 region	
Result	Passed		

Table 5.6: Test case TC-03: Share 1 can be fetched exactly once.

Test Case ID	TC-03	Test Date	November 2026
Test case version	2.0	Use case reference(s)	FR-11, FR-15
Revision history	Supersedes the FYP Part 1 manual check	Verified by	test_image_sharing.py
Objective	Confirm that the capability token is single-use and that integrity analysis does not consume it.		
Product / module	Share endpoint; message store.		

Environment	Flask test client with an authenticated session.		
Assumptions	The token is drawn from secrets.token_urlsafe(32) and is not derived from the message identifier.		
Pre-requisite	An image message exists and the recipient holds the token.		
Step	Execution description	Expected result	
1	Request the integrity analysis for the message	Analysis returns; the access flag is still unset	
2	Fetch the share with the token	The share bytes are returned	
3	Fetch the share again with the same token	The request returns 404	
4	Fetch with a token belonging to another message	The request returns 404	
Result	Passed		

Table 5.7: Test case TC-04: The TOTP attempt limit survives reconnection.

Test Case ID	TC-04	Test Date	November 2026
Test case version	2.0	Use case reference(s)	FR-4, FR-5
Revision history	Supersedes the FYP Part 1 manual check	Verified by	test_auth.py
Objective	Confirm that dropping the socket does not reset the second-factor attempt counter, and that a correct password does not clear it.		
Product / module	Authentication; throttling.		
Environment	Socket.IO test client; several sockets opened in advance.		
Assumptions	Opening the sockets before guessing is essential: a sequential loop is stopped by the login gate and never reaches the check under test.		
Pre-requisite	An account exists with TOTP enrolled and the correct password known.		
Step	Execution description	Expected result	
1	Open more sockets than the attempt ceiling allows	All connect successfully	
2	On each socket, authenticate with the correct password	Each reaches the second-factor stage	
3	Submit a wrong code on each socket in turn	Failures accumulate against the account, not the socket	
4	Continue past the ceiling	Further attempts are refused; the unfixed code permitted 30 against a ceiling of 13	
Result	Passed		

5.5.1 Test metrics

Table 5.8 reports the metrics over the suite as a whole. The two skipped tests are environment-gated rather than failing: they require a configured Firebase service account, which is deliberately absent from the repository.

Table 5.8: Test metrics over the complete suite.

Metric	Definition	Value
Number of test cases	Total developed for the system	485 (416 server, 69 browser)
Number executed	Run in the final verification pass	485
Number passed	Completed with all assertions satisfied	483
Number failed	Completed with an assertion violated	0
Number skipped	Environment-gated, requiring Firebase credentials	2
Test case defect density	(failed x 100) / executed	0.00%
Pass rate	passed / (executed - skipped)	100.00%
Requirements traceability	Functional requirements with a named verifying module	22 of 22 (100%)
Mutation verification	Fixes whose tests were confirmed able to fail	3 of 3; one vacuous test found and rewritten
Suite runtime	Wall clock, server suite	87 seconds
Test code volume	Lines of automated test code	6,307 (5,291 Python, 1,016 JavaScript)

5.6 Results and Discussion

5.6.1 Reconstruction fidelity

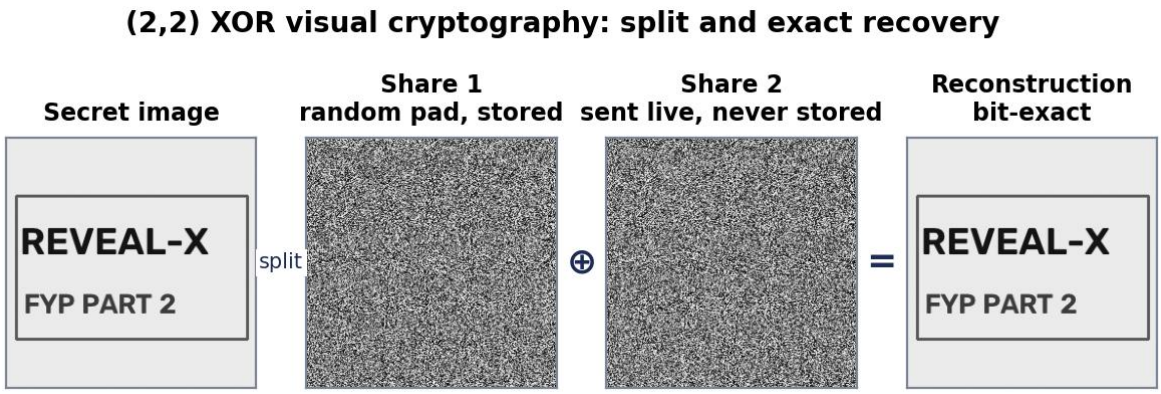


Figure 5.2: The (2,2) XOR pipeline on a 256x256 test image: the secret, the two shares, and the recovered image.

Figure 5.2 shows the pipeline on a document-like test image. Neither share reveals any structure and the recovery is visually identical to the input. Part 1 of this project recorded as a limitation that reconstructed images showed "measurable PSNR and SSIM degradation". That statement is incorrect, and the correction matters because it changes what the enhancement objective was supposed to fix.

Table 5.9: Reconstruction fidelity over six test images. Every reconstruction is bit-exact.

Test image	Size	Bit-exact	MSE	PSNR (dB)	SSIM	Differing pixels
gradient	256x256	Yes	0.0	infinite	1.0000	0
checkerboard	256x256	Yes	0.0	infinite	1.0000	0
document	256x256	Yes	0.0	infinite	1.0000	0

Test image	Size	Bit-exact	MSE	PSNR (dB)	SSIM	Differing pixels
natural-like	256x256	Yes	0.0	infinite	1.0000	0
logo	256x256	Yes	0.0	infinite	1.0000	0
app-icon	256x256	Yes	0.0	infinite	1.0000	0

Table 5.9 reports six images: four synthetic patterns chosen to stress different statistics and two real assets from the application. In every case the recovered array is identical to the input pixel for pixel, so mean squared error is exactly zero, peak signal-to-noise ratio is unbounded and the structural similarity index [25] is exactly 1.0000. This is a property of the scheme rather than a favourable measurement: XOR is its own inverse and there is no rounding step anywhere in the path. Lossy behaviour can enter only through a lossy transport or storage format, which is why shares are handled as PNG and never as JPEG.

5.6.2 Statistical security of the shares

Bit-exact recovery establishes that the scheme is correct. It says nothing about whether a single share leaks the secret, so three independent statistical tests were run.

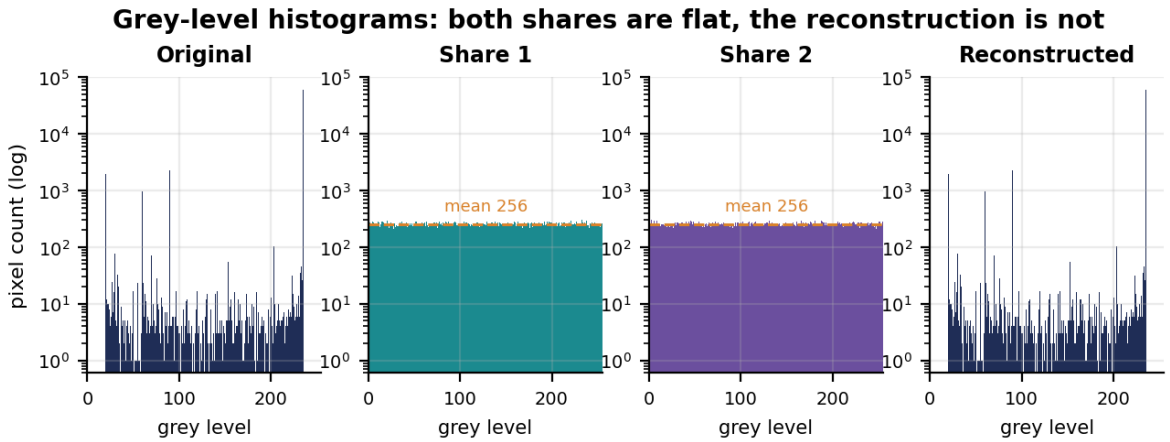


Figure 5.3: Grey-level histograms on a logarithmic count axis. The original and the recovery are strongly peaked; both shares are flat at the uniform expectation.

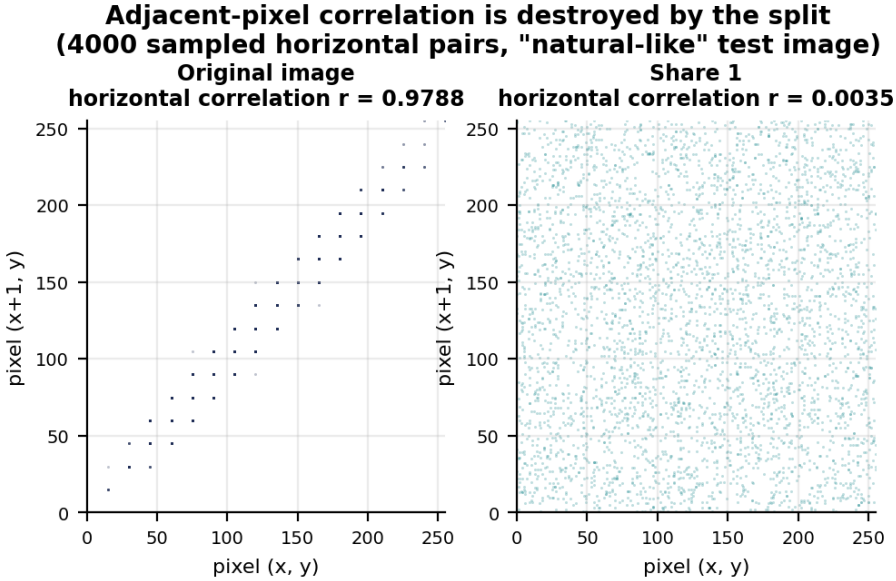


Figure 5.4: Adjacent horizontal pixel pairs, 4000 samples. Natural images lie on the diagonal; a share fills the square uniformly.

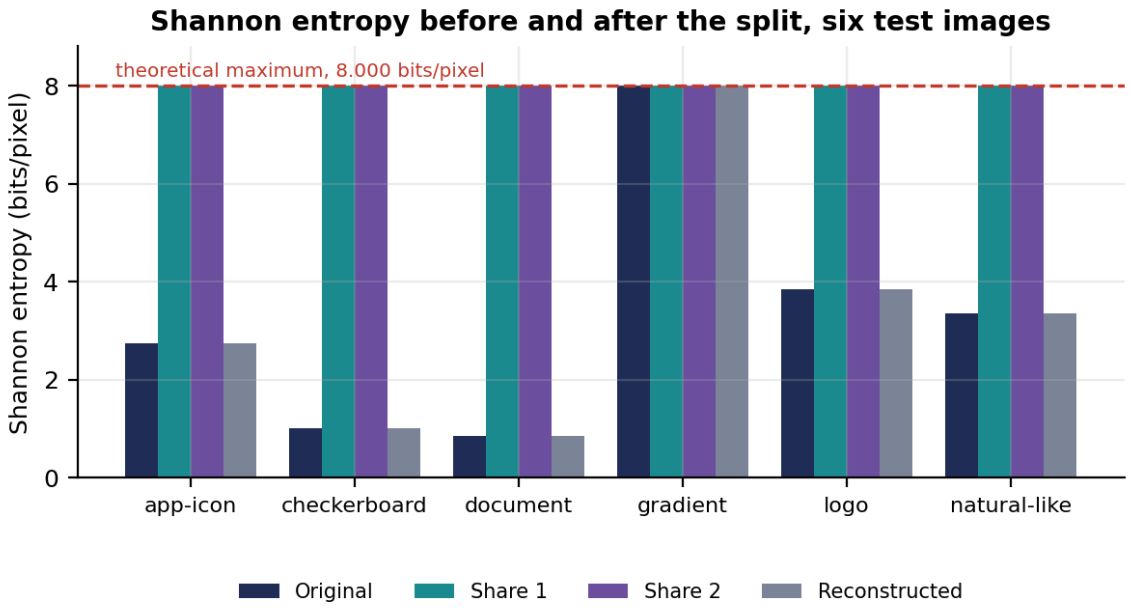


Figure 5.5: Shannon entropy of each array across the six test images. Both shares sit at the 8.000 bits per pixel maximum regardless of the content they conceal.

The first test is the grey-level histogram. A natural or document image concentrates its mass on a few levels; both shares are flat across all 256 levels. A chi-square goodness-of-fit test against the uniform distribution fails to reject uniformity for every one of the twelve share arrays measured, with p values from 0.117 to 0.764, and rejects it decisively for every original except one.

That exception is instructive, and it is a warning against reading too much into a single test. The gradient test image is a linear ramp, so it visits every grey level equally often and its histogram is uniform by construction; the chi-square test cannot distinguish it from random noise at all. Its adjacent-pixel correlation, however, is almost 1. A flat histogram is necessary for a share to look random but nowhere near sufficient, which is exactly why the detector measures correlation and entropy alongside histogram shape.

The second test is adjacent-pixel correlation, the standard check in the image-encryption literature. On the natural-like test image the horizontal correlation of the original is $r = 0.9788$, the strong diagonal expected of real image content; on its Share 1 the same statistic is $r = 0.0035$, statistically indistinguishable from zero at this sample size. The structure an adversary would exploit has been removed, not merely hidden. The third is Shannon entropy [8]: both shares reach the 8-bit maximum to four decimal places on every test image, including the checkerboard whose original entropy is near 1 bit per pixel.

Table 5.10: Statistical profile of the originals, the shares and the recovery for two representative test images.

Test image	Array	Entropy (bits/px)	Mean	Corr. h	Corr. v	Chi-square p	Uniform ?
document	original	0.8494	218.5	0.8962	0.8737	0.0000	no
document	share 1	7.9972	127.5	-0.0037	-0.0092	0.4580	yes
document	share 2	7.9972	127.4	-0.0017	-0.0122	0.5696	yes
document	reconstructed	0.8494	218.5	0.8962	0.8737	0.0000	no
natural-like	original	3.3539	128.4	0.9788	0.9796	0.0000	no
natural-like	share 1	7.9973	127.8	-0.0052	-0.0005	0.6440	yes
natural-like	share 2	7.9973	127.2	-0.0043	-0.0033	0.6486	yes
natural-like	reconstructed	3.3539	128.4	0.9788	0.9796	0.0000	no

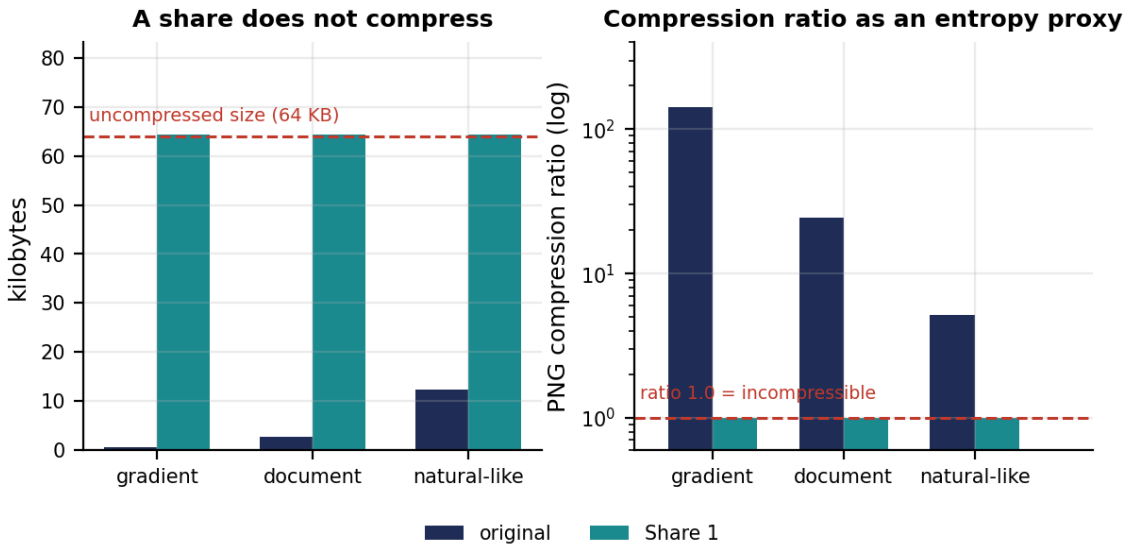


Figure 5.6: Shares do not compress. PNG output size and compression ratio for three originals and their Share 1.

Table 5.11: Storage cost of a share. A share is incompressible, so storing it costs the full uncompressed size of the image.

Test image	Raw bytes	PNG original	PNG Share 1	Original ratio	Share ratio	Storage overhead
gradient	65536	461	65976	142.2x	0.993x	143x
document	65536	2672	65976	24.5x	0.993x	25x
natural-like	65536	12668	65976	5.2x	0.993x	5x

A fourth piece of evidence falls out of the implementation rather than from a deliberate test. PNG uses lossless entropy coding, so its output size is a practical upper bound on compressibility. The gradient image compresses 142-fold; its share compresses by a factor of 0.993, meaning the encoder finds no redundancy at all and the file is slightly larger than the raw pixels because of framing overhead. Incompressibility is an empirical statement about entropy that depends on no statistical model. The engineering consequence is a 143-

fold storage increase in the worst case measured, which is part of why messages expire after one hour.

5.6.3 Tamper detection results

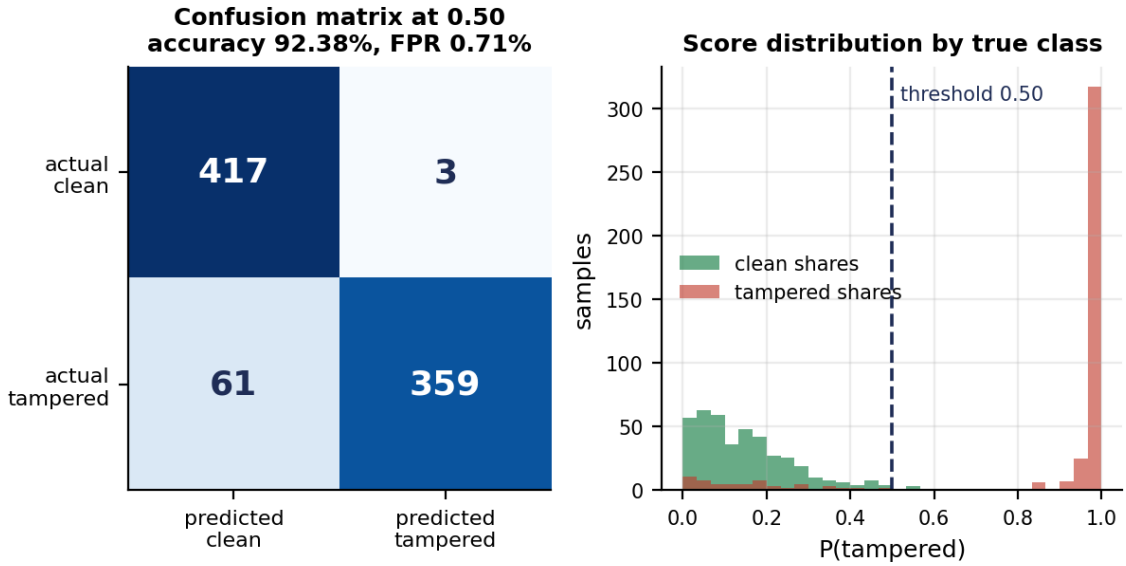


Figure 5.7: Confusion matrix at the shipped 0.50 threshold, and the distribution of predicted probabilities by true class.

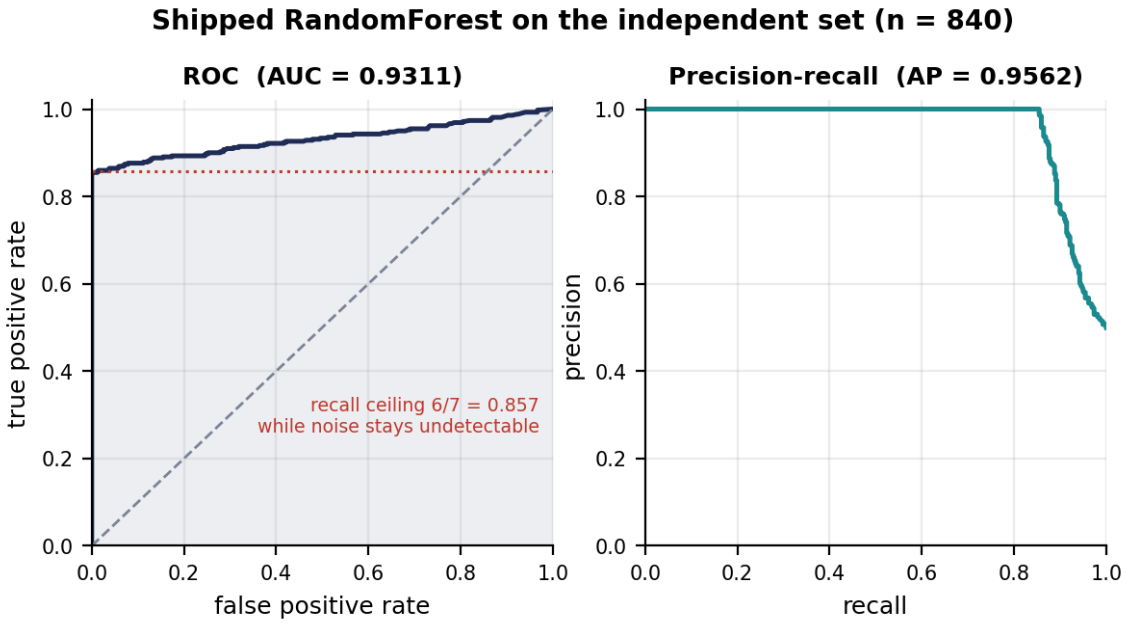


Figure 5.8: ROC and precision-recall curves on the independent set. The dotted line marks the recall ceiling imposed by the noise blind spot.

Table 5.12: Headline detection performance on the independent set (n = 840), at three decision thresholds.

Metric	Threshold 0.50 (shipped)	Threshold 0.35	Threshold 0.60
Accuracy	92.38%	90.00%	92.74%
Precision	99.17%	92.42%	100.00%

Metric	Threshold 0.50 (shipped)	Threshold 0.35	Threshold 0.60
Recall	85.48%	87.14%	85.48%
F1 score	0.9182	0.8971	0.9217
False positive rate	0.71%	7.14%	0.00%
True positives / false negatives	359 / 61	366 / 54	359 / 61
True negatives / false positives	417 / 3	390 / 30	420 / 0

On the independent set the shipped model reaches 92.38% accuracy at its default threshold, with 99.17% precision and 85.48% recall. The area under the ROC curve is 0.9311 and average precision is 0.9562 [26]. Of 420 clean shares, three were misclassified, a false positive rate of 0.71%.

The score distribution in Figure 5.7 is more informative than the accuracy figure. The two classes are almost perfectly separated: tampered shares pile up at a probability of 1.0 and clean shares at 0.0, with very little mass between. The 61 errors are not borderline cases that a better threshold would recover; they are a specific, identifiable subset.

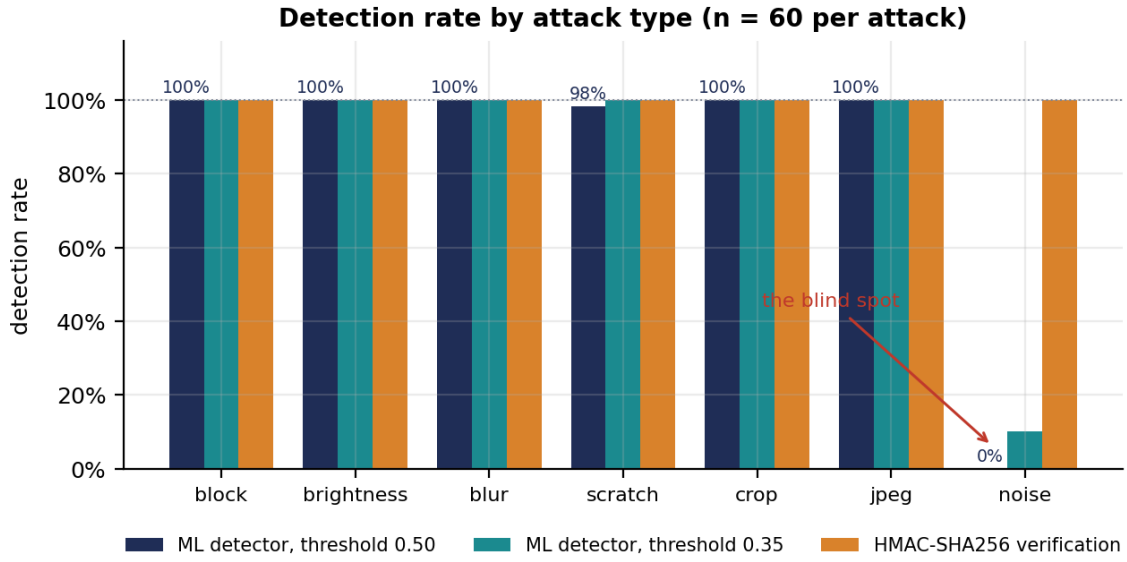


Figure 5.9: Detection rate by attack type at two thresholds, against the HMAC verification rate on the same attacks.

Table 5.13: Per-attack detection on the independent set. Six attacks are detected essentially without error; one is not detected at all.

Attack	n	Mean P(tampered)	Detected at 0.50	Rate at 0.50	Detected at 0.35	Rate at 0.35
block	60	1.000	60 / 60	100.0%	60 / 60	100.0%
brightness	60	0.995	60 / 60	100.0%	60 / 60	100.0%
blur	60	0.987	60 / 60	100.0%	60 / 60	100.0%
scratch	60	0.950	59 / 60	98.3%	60 / 60	100.0%
crop	60	1.000	60 / 60	100.0%	60 / 60	100.0%
jpeg	60	0.968	60 / 60	100.0%	60 / 60	100.0%

Attack	n	Mean P(tampered)	Detected at 0.50	Rate at 0.50	Detected at 0.35	Rate at 0.35
noise	60	0.158	0 / 60	0.0%	6 / 60	10.0%
clean (false positives)	420	0.149	3 / 420	0.7%	30 / 420	7.1%

Aggregate accuracy hides the structure of the failures. Six of the seven attacks are detected at or very near 100%, with mean predicted probability from 0.950 to 1.000. The seventh, random-noise injection, is detected in 0 of 60 trials at the shipped threshold and accounts for 60 of the 61 false negatives; the remaining one is a single missed scratch. Almost the entire error budget of this classifier is one attack type.

That yields a useful analytical check. If the noise attack were undetectable in principle and everything else were classified perfectly with no false positives, accuracy on this dataset would be 0.9286. The measured accuracy is 0.9238 at the shipped threshold and 0.9274 at 0.60, within 0.1 percentage points of that ceiling. The classifier is therefore not under-trained: it performs at close to the maximum its feature set permits, and the remaining gap is a representation problem rather than a learning problem.

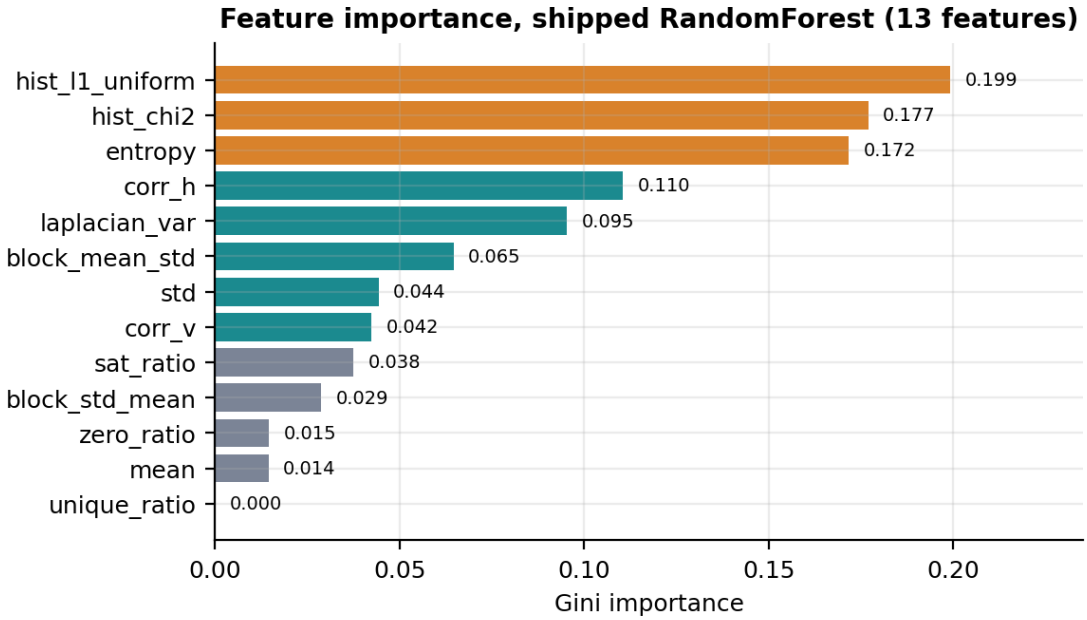


Figure 5.10: Gini importance of the thirteen features in the shipped RandomForest.

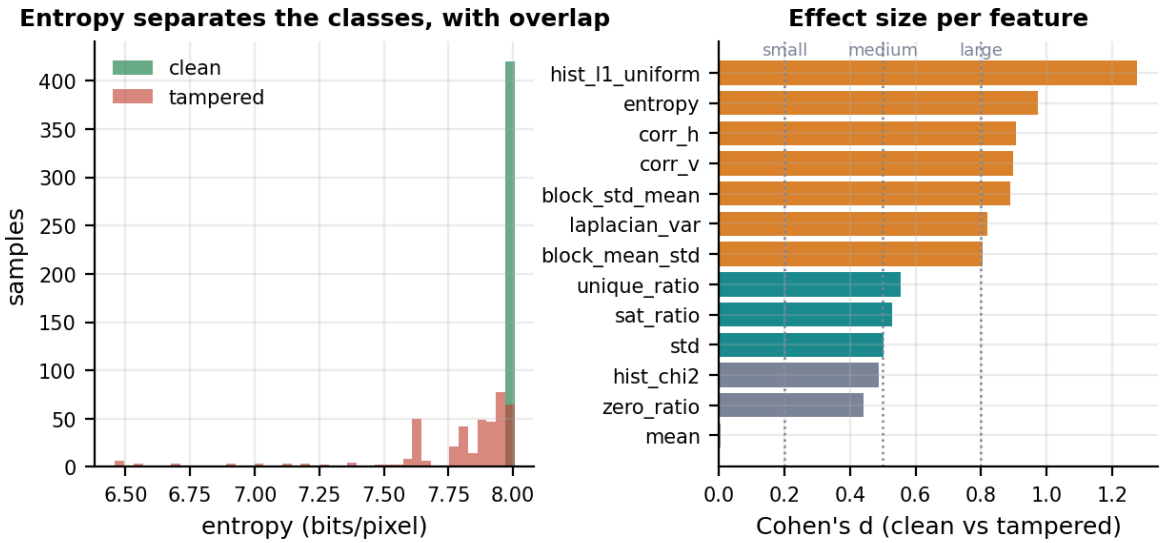


Figure 5.11: Left: the entropy distribution of clean and tampered shares. Right: effect size per feature, measured as Cohen's d.

Table 5.14: The eight most important features, with effect size and significance measured independently of the model on the evaluation set.

Rank	Feature	Gini importance	Clean mean	Tampered mean	Cohen's d	Mann-Whitney p
1	hist_l1_uniform	0.1993	0.1321	0.2749	1.277	4.0e-101
2	hist_chi2	0.1770	0.9915	49.3952	0.487	6.1e-104
3	entropy	0.1717	7.9800	7.7800	0.974	1.2e-103
4	corr_h	0.1104	-0.0012	0.1139	0.907	1.7e-31
5	laplacian_var	0.0953	109771.6	91210.6	0.819	9.5e-20
6	block_mean_std	0.0646	4.4816	10.6727	0.804	6.0e-37
7	std	0.0443	73.8858	70.5187	0.503	1.2e-08
8	corr_v	0.0423	-0.0002	0.1147	0.899	6.9e-31

The top three features account for 54.8% of total Gini importance and all three measure the same underlying quantity from different directions: how far the histogram has moved from uniform. Because a random forest distributes credit arbitrarily among correlated features, the effect sizes in Table 5.14 are computed independently of the model. They agree with the importance ranking, with Cohen's d of 1.28 for hist_l1_uniform and 0.97 for entropy, both above the 0.8 conventionally called a large effect, and a Mann-Whitney U test rejecting equality at $p < 1e\text{-}30$ for the leading features. The entropy panel of Figure 5.11 also explains the blind spot visually: clean shares form a tight spike at the 8.000-bit maximum, and an attack that leaves entropy at the maximum lands inside the clean cluster.

5.6.4 The noise blind spot

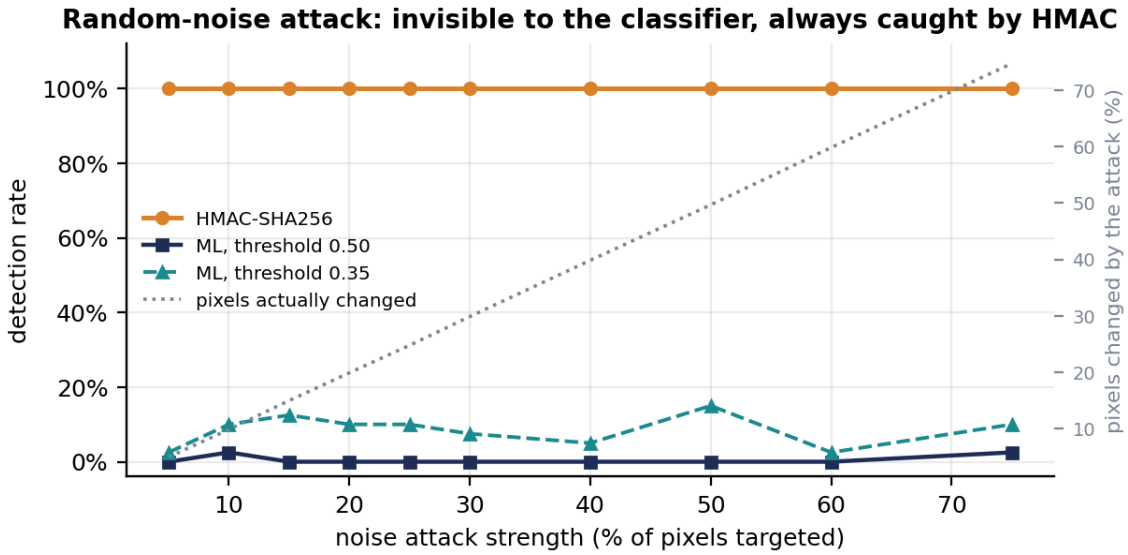


Figure 5.12: Noise attack across ten strengths: detection by the classifier and by HMAC, with the fraction of pixels the attack actually changed.

Table 5.15: The noise attack swept from 5% to 75% strength, 40 trials per level. The attack is real at every level and invisible to the classifier at every level.

Strength	Pixels changed	Mean abs. delta	Mean P(tampered)	ML rate at 0.50	ML rate at 0.35	HMAC rate
5%	5.0%	4.3	0.145	0.0%	2.5%	100.0%
10%	9.9%	8.5	0.156	2.5%	10.0%	100.0%
15%	15.0%	12.8	0.176	0.0%	12.5%	100.0%
20%	19.9%	17.0	0.137	0.0%	10.0%	100.0%
25%	24.8%	21.3	0.157	0.0%	10.0%	100.0%
30%	29.9%	25.6	0.156	0.0%	7.5%	100.0%
40%	39.9%	34.1	0.167	0.0%	5.0%	100.0%
50%	49.7%	42.4	0.169	0.0%	15.0%	100.0%
60%	59.8%	51.4	0.152	0.0%	2.5%	100.0%

Strength	Pixels changed	Mean abs. delta	Mean P(tampered)	ML rate at 0.50	ML rate at 0.35	HMAC rate
75%	74.7%	64.0	0.168	2.5%	10.0%	100.0%

A single failing attack deserves to be understood rather than merely reported. The first hypothesis was that the training strength range did not cover the cases being tested, which a wider sweep would fix. The sweep in Table 5.15 refutes it: detection never exceeds 2.5% at the shipped threshold anywhere, and the mean predicted probability stays between 0.137 and 0.176 throughout, essentially identical to the 0.149 the model assigns to genuinely clean shares. Strength is not the variable that matters.

The attack is unquestionably occurring: the measured fraction of changed pixels tracks the requested strength almost exactly, up to 74.7% of all pixels with a mean absolute change of 64 grey levels. Three quarters of the image is overwritten and the classifier still sees nothing.

The explanation is structural and generalises beyond this implementation. The feature vector measures departures from uniform randomness. The noise attack replaces uniform random pixels with fresh uniform random pixels. The result is still uniform random, so every one of the thirteen features is unchanged in distribution and the feature vector is, by construction, that of a clean share. No quantity of additional training data and no alternative classifier over these features can fix this, because the two classes are genuinely identical in this representation.

This is the most useful result in the evaluation, and the reason is worth stating plainly: a system with only a machine-learned integrity check would be fully exploitable by an attacker who knew this, and the exploit is trivial. The next section shows why Reveal-X is not.

5.6.5 Two-layer integrity

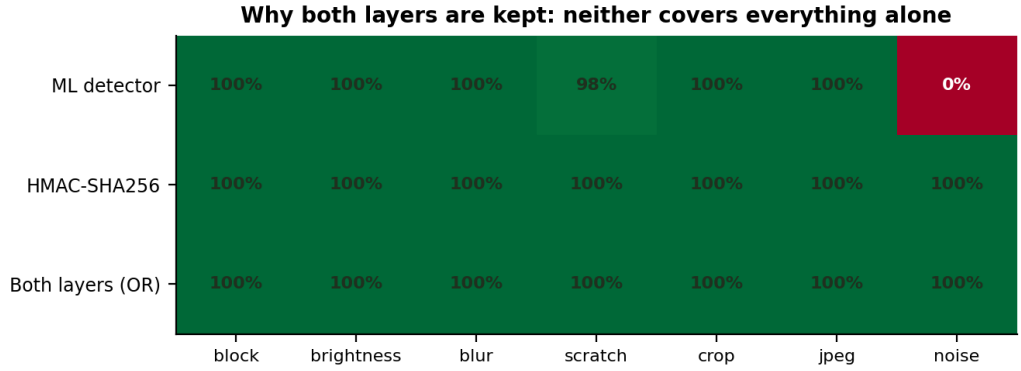


Figure 5.13: Coverage matrix. Neither layer detects every attack; the union of the two does.

Table 5.16: The two integrity layers compared on identical attacks.

Attack	ML detected	ML rate	HMAC detected	HMAC rate	Either layer
block	60 / 60	100.0%	60 / 60	100.0%	100.0%
brightness	60 / 60	100.0%	60 / 60	100.0%	100.0%
blur	60 / 60	100.0%	60 / 60	100.0%	100.0%
scratch	59 / 60	98.3%	60 / 60	100.0%	100.0%
crop	60 / 60	100.0%	60 / 60	100.0%	100.0%
jpeg	60 / 60	100.0%	60 / 60	100.0%	100.0%
noise	0 / 60	0.0%	60 / 60	100.0%	100.0%
clean shares accepted	—	—	120 / 120	100.0%	—

Each share is stored with an HMAC-SHA256 tag [11] computed over the bytes as written to disk and compared on retrieval in constant time. HMAC detects 420 of 420 tampered shares across all seven attacks, including 60 of 60 noise attacks, and accepts 120 of 120 untouched shares. This is the expected behaviour of a secure MAC rather than a surprising result: any change to any byte changes the tag.

The layers therefore fail in different places, which is the entire argument for carrying both, and the hypothesis stated in Section 4.2.2 is confirmed. HMAC is exact but says only that something changed — no location, no classification, and no answer at all once a share has been legitimately re-encoded, since a JPEG round trip breaks the tag as thoroughly as an attack does. The classifier survives re-encoding, localises damage to a region and explains its verdict, but is blind to the one attack HMAC handles most easily. Neither is redundant.

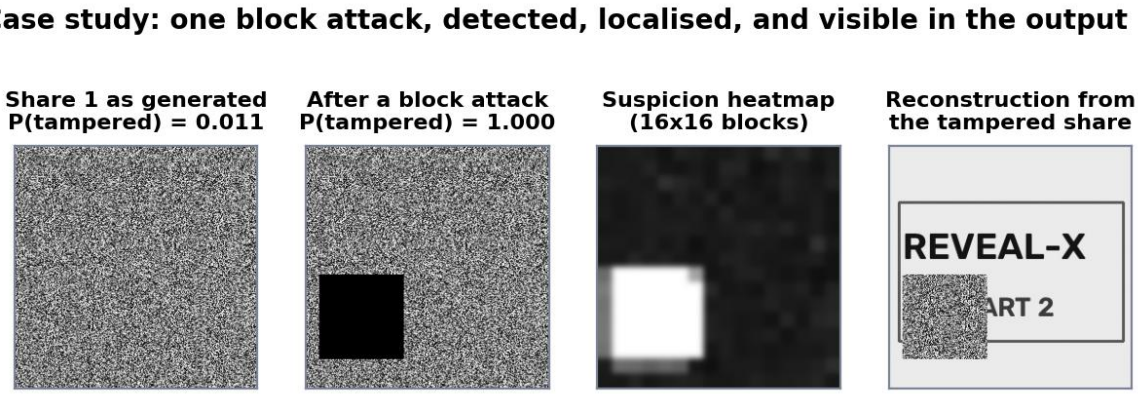


Figure 5.14: One block attack end to end: the clean share, the tampered share, the suspicion heatmap, and the visibly corrupted reconstruction.

Figure 5.14 follows a single block attack through the pipeline. The clean share scores P(tampered) = 0.0111; after the attack the same share scores 1.0000, the 16x16 block heatmap localises the damage to the correct region, and the HMAC check fails. The detector also returns the reasons behind its verdict in readable form — that entropy is lower than a valid random share, that the pixel mean has shifted from the expected centre, that neighbouring pixels show unusual correlation, and that large numbers of pure black or white pixels indicate block replacement or clipping.

5.6.6 Operating point and model selection

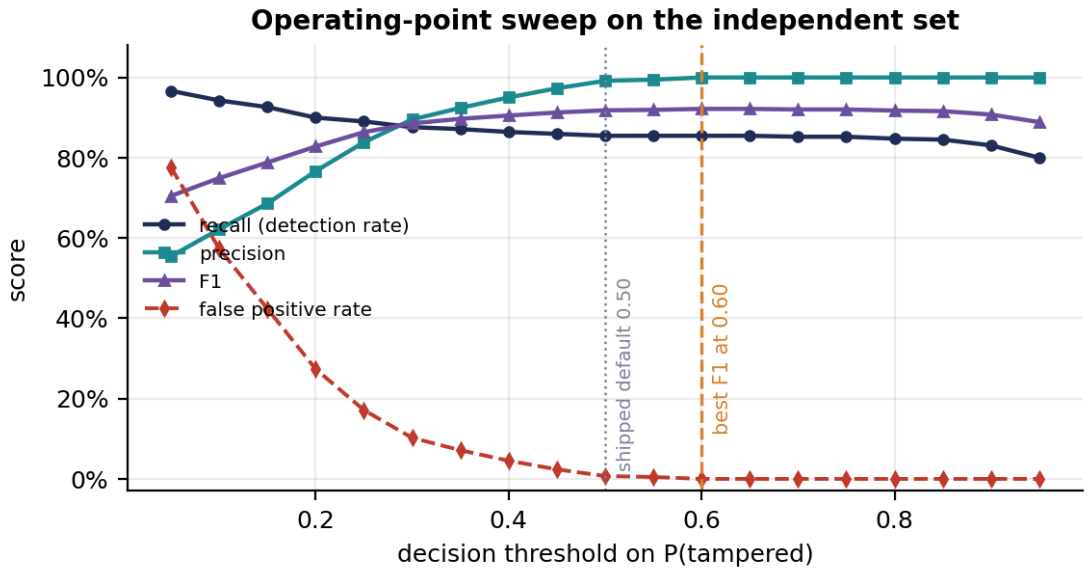


Figure 5.15: Recall, precision, F1 and false positive rate against the decision threshold.

Table 5.17: Operating points. Raising the threshold from 0.50 to 0.60 removes every false positive without costing a single detection.

Threshold	Accuracy	Precision	Recall	F1	FPR	TP	FP	FN	TN
0.10	68.45%	62.17%	94.29%	0.7493	57.38%	396	241	24	179

Threshold	Accuracy	Precision	Recall	F1	FPR	TP	FP	FN	TN
0.20	81.31%	76.67%	90.00%	0.8280	27.38%	378	115	42	305
0.30	88.69%	89.54%	87.62%	0.8857	10.24%	368	43	52	377
0.35	90.00%	92.42%	87.14%	0.8971	7.14%	366	30	54	390
0.40	90.95%	95.03%	86.43%	0.9052	4.52%	363	19	57	401
0.50	92.38%	99.17%	85.48%	0.9182	0.71%	359	3	61	417
0.60	92.74%	100.00%	85.48%	0.9217	0.00%	359	0	61	420
0.70	92.62%	100.00%	85.24%	0.9203	0.00%	358	0	62	420
0.80	92.38%	100.00%	84.76%	0.9175	0.00%	356	0	64	420
0.90	91.55%	100.00%	83.10%	0.9077	0.00%	349	0	71	420

The shipped threshold of 0.50 is the library default rather than a choice the project made, so it was worth testing. The result is an unambiguous recommendation: at 0.60 the model achieves 100% precision with identical recall, eliminating all three false positives and raising accuracy to 92.74%, which is 0.0012 short of the analytical ceiling derived in Section 5.6.3. Lowering the threshold to 0.35 is the opposite trade — recall improves by 1.7 percentage points while false positives rise from 3 to 30 — and for an integrity warning shown to a user that is the wrong direction, because a false alarm on a clean share costs trust and the recall it buys consists of noise attacks that sit nowhere near the boundary.

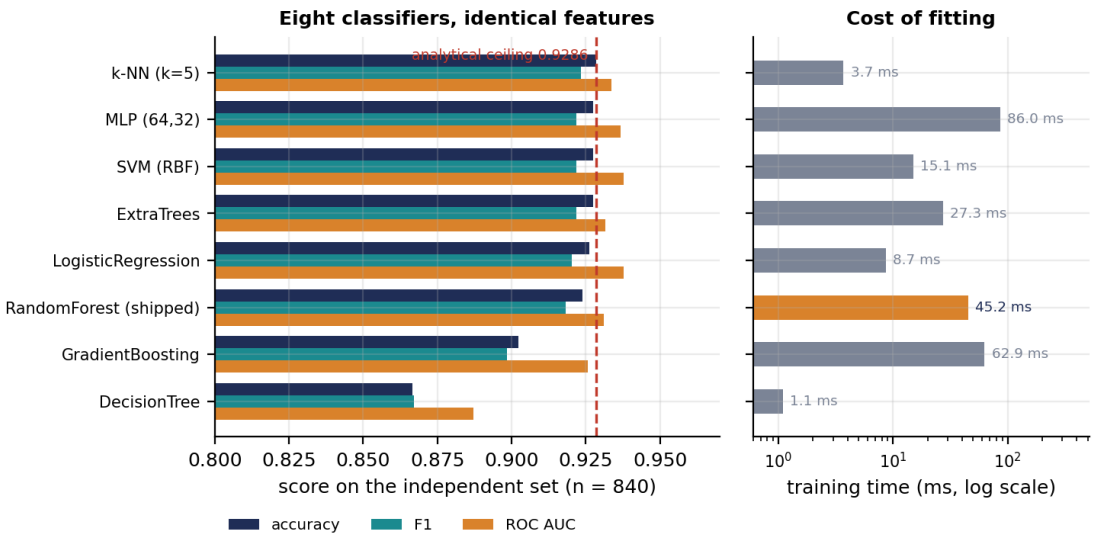


Figure 5.16: Eight classifiers on identical features. All cluster at the ceiling imposed by the blind spot.

Table 5.18: Eight classifiers trained on identical features and the same split, evaluated on the same independent set, with the rule-based fallback for reference.

Classifier	5-fold CV accuracy	Internal test	D_eval accuracy	Precision	Recall	F1	ROC AUC
k-NN (k=5)	0.921 ± 0.036	0.933	0.9286	0.997	0.860	0.923	0.9335
ExtraTrees	0.929 ± 0.031	0.950	0.9274	1.000	0.855	0.922	0.9315
SVM (RBF)	0.917 ± 0.037	0.950	0.9274	1.000	0.855	0.922	0.9378

Classifier	5-fold CV accuracy	Internal test	D_eval accuracy	Precision	Recall	F1	ROC AUC
MLP (64,32)	0.929 ± 0.031	0.950	0.9274	1.000	0.855	0.922	0.9366
LogisticRegression	0.929 ± 0.031	0.950	0.9262	1.000	0.852	0.920	0.9378
RandomForest (shipped)	0.925 ± 0.036	0.933	0.9238	0.992	0.855	0.918	0.9311
GradientBoosting	0.921 ± 0.024	0.950	0.9024	0.936	0.864	0.899	0.9257
DecisionTree	0.867 ± 0.021	0.900	0.8667	0.865	0.869	0.867	0.8871
Statistical fallback (no model file)	—	—	0.6786	1.000	0.357	0.526	0.9048

A reviewer is entitled to ask whether a different model would do better. Eight classifiers spanning linear, kernel, instance-based, tree and neural families were trained on the same features and the same split. The spread is narrow: seven land between 0.9024 and 0.9286, and the best beats the shipped forest by 0.48 percentage points, a difference of four samples out of 840 and well inside sampling noise. Only the single decision tree is clearly worse, as expected of an unensembled learner on thirteen correlated features.

That clustering is the real finding and confirms the prediction of Section 5.6.4: every model converges on the same ceiling because every model reads the same thirteen numbers, and in that representation the noise attack is not present. The limitation lives in the features, not the learner, so changing the learner is not where effort should go. The RandomForest is nevertheless the right thing to ship for reasons the accuracy column does not show: it exports cleanly to ONNX for in-browser execution, it yields the per-feature importances of Section 5.6.3, and it produces the human-readable explanations the interface displays. The last row quantifies what the machine-learning component contributes over the hand-tuned fallback that runs when the model file is absent: 24.5 percentage points of accuracy and 49.8 points of recall.

5.6.7 Objective 9: not delivered, and why

Objective 9 specified a denoising convolutional neural network and ESRGAN super-resolution applied to the reconstructed image. Neither was implemented. The enhancement stage in the codebase remains a classical pipeline of bilateral filtering, unsharp masking and contrast-limited adaptive histogram equalisation. This section states the reason rather than leaving the omission unexplained.

The objective was written on the assumption recorded in Part 1 and following the enhancement approach surveyed there [40]: that reconstruction was lossy and the recovered image needed restoring. Section 5.6.1 shows that assumption to be false. Reconstruction is bit-exact, with mean squared error of exactly zero and structural

similarity of exactly 1.0000 on every test image. A denoising network such as DnCNN [30], trained by the standard supervised procedure [38], learns a mapping from a degraded image to a clean one; here the input and the target are the same array, so the optimal learned mapping is the identity and the training objective is degenerate. There is nothing for the network to remove.

The same reasoning applies with less force to the super-resolution half. ESRGAN [32], building on SRGAN [31], upscales plausibly rather than faithfully: it synthesises detail that was not present. For a security application whose purpose is to establish whether an image has been altered, adding invented detail to the output is actively undesirable, because a generative hallucination would be indistinguishable from genuine tampering. The project does downscale images above 500 pixels on the long edge, so there is a real case for upscaling on display, but it would have to be marked clearly as a reconstruction rather than as evidence.

A defensible route exists if the milestone must be completed, and it is costed here rather than described vaguely. The project's own attack simulator already produces matched damaged and clean pairs on demand, which is exactly the supervised training data a restoration network needs. Training a small residual CNN on those pairs, with PyTorch as a development-only dependency, and exporting it to ONNX alongside the existing classifier would reuse the browser inference path that Section 5.6.8 already validates. The honest framing of that work is repair of a tampered share, not denoising of a clean reconstruction — and under that framing it is a genuinely useful extension rather than a solution without a problem.

5.6.8 Browser and server parity

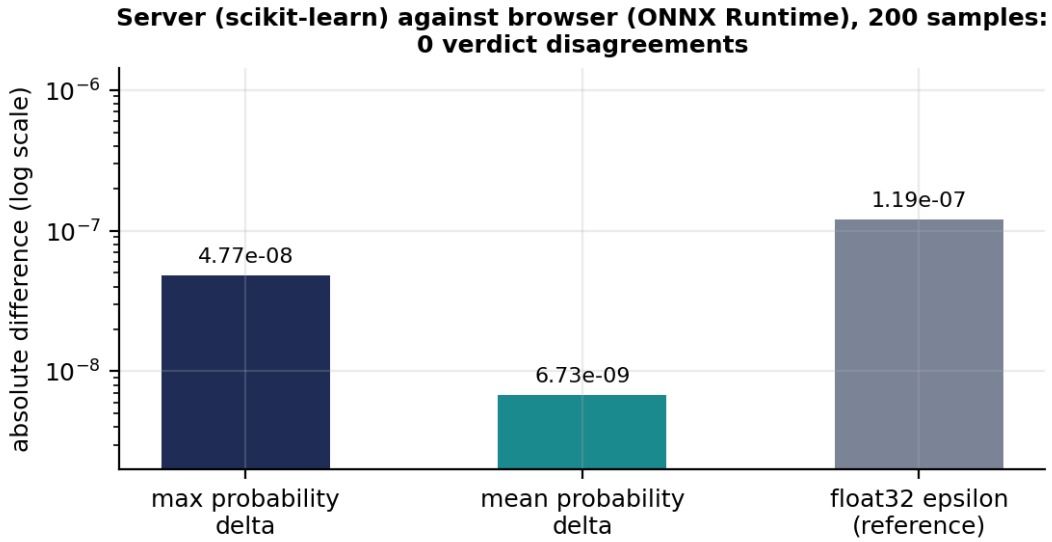


Figure 5.17: Agreement between the scikit-learn model on the server and the ONNX model in the browser over 200 samples.

Table 5.19: Server-to-browser agreement. The model and the feature extractor were verified separately.

Property	Measured	Interpretation
Samples compared	200	Mixed clean and tampered, all seven attacks
Maximum probability delta	4.768e-08	Below float32 epsilon (1.19e-07): a representation effect, not a model difference
Mean probability delta	6.734e-09	Two orders of magnitude smaller again
Verdict disagreements	0	The two runtimes never reach opposite conclusions
Browser inference time	0.0125 ms	ONNX Runtime, single sample
Server inference time	1.8875 ms	scikit-learn, single sample, dominated by Python call overhead
Feature parity fixture	pass	Ten reference cases agree within 1e-12 relative

A model that disagreed with itself across runtimes would be worse than having no second opinion at all, so agreement is verified rather than assumed. Over 200 samples the largest difference in predicted probability is 4.77e-08, below the 1.19e-07 epsilon of 32-bit floating point: the ONNX export uses float32 inputs while scikit-learn computes in float64, and the measured gap is exactly the size that difference predicts. There are zero verdict disagreements.

Model agreement alone is insufficient, because the browser must also reproduce the thirteen features from raw pixels using an independent JavaScript implementation. A fixture of ten reference cases, generated on the server and checked into the repository, holds that path in place, and the check passes within a relative tolerance of 1e-12. An earlier byte-exact version of the check proved too strict: two correlation features differed in the fifteenth decimal place purely because of floating-point summation order, which is a

useful illustration of why numerical tolerance must be chosen deliberately rather than set to zero.

5.6.9 Performance

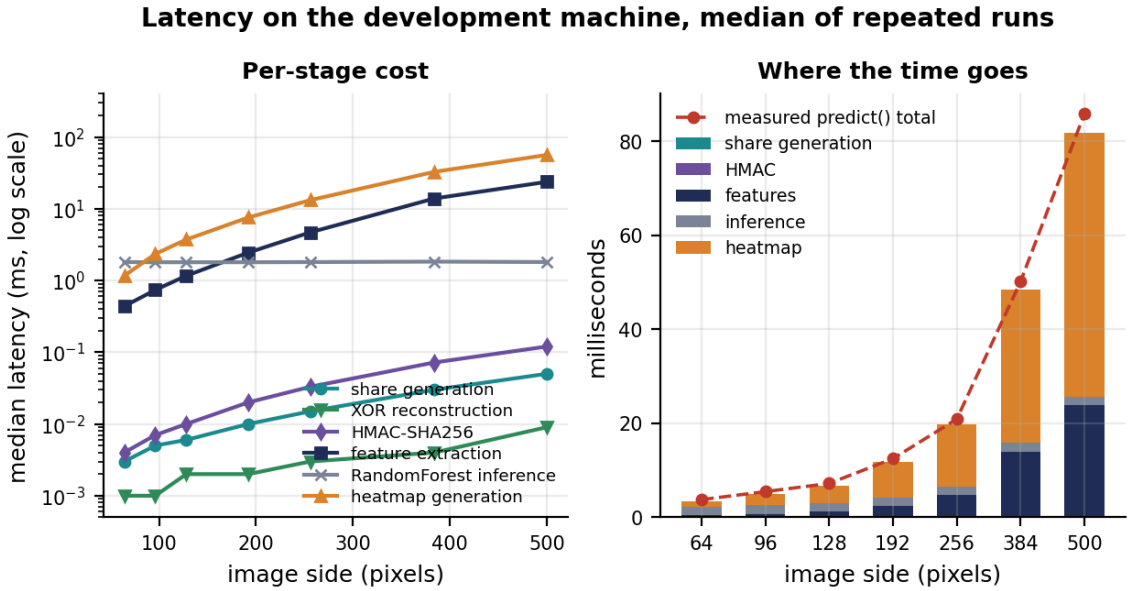


Figure 5.18: Per-stage latency against image size, and the composition of the total detection cost.

Table 5.20: Median latency in milliseconds by stage and image size. The cryptographic operations are negligible; the heatmap dominates.

Image side	Pixels	Share gen.	XOR recon.	HMAC	Features	Inference	Heatmap	Full predict()
64	4096	0.003	0.001	0.004	0.44	1.81	1.2	3.8
96	9216	0.005	0.001	0.007	0.74	1.80	2.3	5.5
128	16384	0.006	0.002	0.010	1.17	1.80	3.7	7.2
192	36864	0.010	0.002	0.020	2.45	1.80	7.6	12.5
256	65536	0.015	0.003	0.033	4.69	1.81	13.2	20.9
384	147456	0.030	0.004	0.072	13.90	1.83	32.7	50.1
500	250000	0.050	0.009	0.120	23.70	1.81	56.2	85.8

The cryptographic core is effectively free. At 256x256, generating both shares takes 0.015 ms, XOR reconstruction 0.003 ms and the HMAC 0.033 ms — three operations costing together less than a twentieth of a millisecond. Visual cryptography is not the expensive part of this system, which is worth stating because the opposite is commonly assumed.

Detection is where the time goes: 4.69 ms to extract features and 1.81 ms to run the forest at 256x256. The full prediction call measures 20.9 ms, and the difference is almost entirely the 13.2 ms heatmap, which evaluates per-block statistics over the whole image. Scaling is roughly linear in pixel count, as expected of per-pixel statistics: from 64 to 500 pixels on a side the pixel count grows 61-fold while full detection grows 23-fold. At the 500-pixel cap

the application enforces, a complete check costs 86 ms, below the threshold at which a user perceives delay and acceptable for a check that runs once per image rather than per frame. If it ever needs to be cheaper, the heatmap is the obvious thing to compute lazily, since it is needed only when the verdict is "tampered".

Three further optimisations were tried during final tuning and discarded on the evidence, which is recorded here because negative results are results. Pooling database connections saves nothing, because a bare connection costs 0.21 ms. Skipping the retention sweep on each message saves 1% of send time. A composite index for the conversation query measured at 0.98 times the original, that is no change, because a multi-index OR plan cannot be pre-sorted and the temporary B-tree for ordering remains regardless. Profiling showed the real cost to be the commit itself at 72%, which is the filesystem sync; write-ahead logging, the usual remedy, made writes 60% slower here because each operation opens its own short-lived connection. One change was kept: a directory-creation call that ran on every connection is now performed once, worth about 4% of send time.

5.7 Best Practices and Coding Standards

5.7.1 Code validation

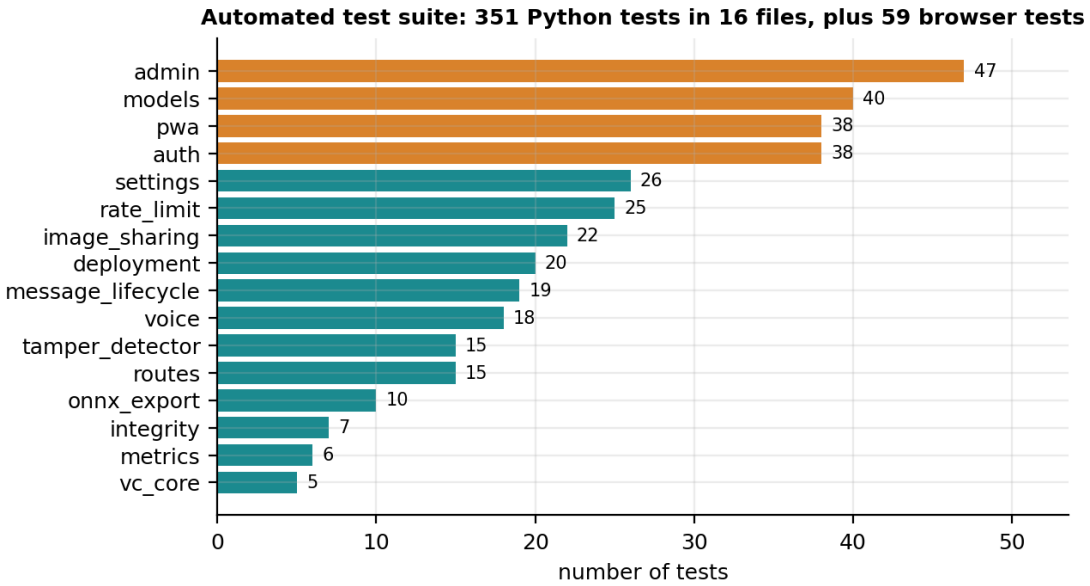


Figure 5.19: Composition of the automated test suite by area.

Validation rests on the 485 automated tests inventoried in Table 5.3, all of which run on every change through continuous integration across two Python versions, a container build and a production-server smoke test. The distribution of those tests is deliberate and reflects where a defect would be most costly: the three largest modules cover administration, the

storage layer and authentication, which are the places where a mistake becomes a privilege escalation rather than a cosmetic fault.

Two browser-side modules test against external criteria rather than against expectations the project set itself. The sanitisation tests exercise the escaping path against the evasion patterns catalogued by OWASP [35], [41], and the accessibility audit measures contrast against the WCAG 2.1 Level AAA thresholds [36] across 114 elements in both themes rather than asserting conformance. That audit is worth a sentence because it changed a claim: it found eight elements in the light theme below even Level AA, including a selected account row at 1.30:1 that was effectively invisible, so the "WCAG 2.1 AA" comment already present in the stylesheet was wrong when it was written. The cause was structural — a translucent fill has no fixed contrast, because contrast depends on what sits behind it — and the fix was to give semantic buttons and the selected row opaque fills.

Table 5.21 gives the size of the implementation by component. The ratio in the last rows is the point rather than an accident: for a security-oriented project, test code comparable in size to application code is a deliberate choice.

Table 5.21: Size of the implementation by component.

Component	Lines of code
Visual cryptography core (vc_core/)	97
AI and ML security (ai_security/)	540
Server routes and storage (routes.py, models.py)	2,883
Authentication (auth/)	302
Training and export scripts	415
Browser cryptography, chat and in-browser ML	5,930
Automated tests (Python and JavaScript)	6,307
Total measured	16,474

5.7.2 Development practices and standards

Six practices governed development and are recorded because they shaped the result rather than merely describing it.

- **Fail closed.** Where a security control cannot complete, the operation is refused rather than degraded. The storage layer refuses a write it cannot prove safe; the browser refuses to score a colour image rather than returning a plausible number computed on the wrong pixels.
- **Measure before optimising, and report negative results.** The three discarded optimisations in Section 5.6.9 cost time and produced nothing shippable. Recording

them prevents the same hypotheses being retried and documents why the obvious remedies do not apply here.

- **Mutation-test the tests.** A test that cannot fail proves nothing. Every assertion added during the final hardening pass was verified by deliberately breaking the code it guards, which found one vacuous test.
- **Pin what must not move, and record why.** The dependency manifest carries comments naming the specific advisories and incompatibilities that forced particular pins, so a future upgrade has the reasoning rather than only the version number.
- **Keep secrets out of the repository and separate by purpose.** All configuration is environment-supplied, and keys serving different purposes are derived separately, as described in Section 5.1.
- **State limitations at the point they arise.** The storage overhead in Section 5.6.2, the absent diffusion in Section 4.2.1 and the blind spot in Section 5.6.4 are recorded where a reader meets them, not collected at the end where they are easy to miss.

5.8 Chapter Summary

This chapter has reported the implementation and its measurement. Reconstruction is bit-exact on every test image, and the three statistical tests confirm that a single share is indistinguishable from uniform random data. The two integrity layers between them detect every one of 420 simulated attacks: the classifier catches six of seven classes at or near 100% with a 0.71% false-positive rate, and the HMAC catches the seventh, which the classifier cannot see at any strength because the attack preserves the feature representation exactly. Eight classifier families converge on the same ceiling, confirming that the limitation is in the features rather than the learner. Server and browser verdicts agree without exception, and a full integrity check costs 86 ms at the size cap the application enforces.

The chapter has also recorded what did not work: one objective not delivered and the measurement that justifies that decision, three performance hypotheses refuted, a vacuous test found by mutation, and an accessibility claim that was wrong when it was first written. Chapter 6 draws these together into the achievements, a critical review and the recommendations for future work.

Chapter 6:

Conclusion and Future Work

Chapter 6: Conclusion and Future Work

6.1 Introduction

This chapter closes the report. It states what the project achieved against the objectives set in Section 1.5, reviews critically what it did not get right, and sets out recommendations for future work. The critical review is given the same weight as the achievements, because a final-year project that reports only its successes has not been evaluated.

6.2 Achievements and Improvements

Nine of the ten objectives were delivered. A (2,2) XOR visual cryptography scheme was implemented with a cryptographically seeded mask drawn afresh for every message, and reconstruction was shown to be bit-exact on every test image, with mean squared error of exactly zero and structural similarity of exactly 1.0000. Three independent statistical tests established that a single share is indistinguishable from uniform random data, with entropy at the 8.000 bits per pixel bound and adjacent-pixel correlation falling from 0.9788 on a natural image to 0.0035 on its share.

Share integrity is enforced by two layers that together detected every one of 420 simulated attacks. The machine-learned layer reaches 92.38% accuracy with 99.17% precision on an independent set fourteen times the size of the internal test split, localises damage to a 16x16 block heatmap, explains its verdict in readable terms, and runs in the recipient’s browser as well as on the server with zero verdict disagreements. Messaging is genuinely end-to-end encrypted with private keys that cannot be extracted from the device that generated them, which converts the key-generation demonstration of the first phase into a working channel.

Three improvements went beyond the stated objectives. The authentication stack gained a three-rank authorisation model with an audit trail, an optional time-based second factor, and throttling on two independent buckets. The interface was audited against WCAG 2.1 Level AAA contrast thresholds across 114 elements in both themes rather than merely claiming conformance, and the audit corrected a claim that was wrong when written. And the verification apparatus grew to 485 automated tests across twenty-four modules, running on every change in continuous integration.

The achievement the project would defend most strongly is not any single number but the discipline behind them. Every figure in Chapter 5 is produced by a script that can be re-run

against the repository. Where a measurement contradicted an earlier claim — as it did for reconstruction quality — the earlier claim was corrected rather than quietly dropped.

6.3 Critical Review

Six weaknesses are recorded here. The first three are design limitations, the next two are process failures, and the last is a limitation of the evaluation itself.

- **Images are not end-to-end encrypted.** The browser uploads the plaintext image and the server performs the split, so the image is in server memory at that moment. The guarantee offered is genuine but narrower than the project originally claimed: nothing the server persists is sufficient to reconstruct the image. Closing the gap properly means splitting in the browser and uploading only Share 1, which is a real design change rather than a configuration one.
- **There is no forward secrecy.** ECDH key pairs are long-lived, so compromising one participant's private key exposes every past message in that conversation that an attacker had captured. Signal has had the established remedy for over a decade and this project does not implement it.
- **The classifier is defeated by an adaptive adversary.** An attacker who knows the thirteen features can construct an attack that preserves all of them, and the noise attack is an accidental instance of exactly that. The HMAC layer is what makes the system safe, and no result in this report bounds classifier performance against an adaptive adversary.
- **The first-phase documentation was inaccurate and was carried forward uncorrected.** Chapters describing the architecture asserted a client-side split, scrypt password hashing and a demonstration-only key exchange, none of which matched the implementation. The error survived into a later revision because the new results were appended rather than reconciled with what was already there. Appending is cheaper than reconciling and it is how documentation becomes untrue.
- **Security testing was performed by the project team.** The assessment in Section 5.4 found and fixed three real issues, but a team testing its own design shares that design's blind spots. Two of the three findings concerned controls the team had itself written and believed to be correct, which is the argument for independent review rather than against the exercise.

- **The evaluation rests on synthetic data from one machine.** All training and evaluation uses simulated attacks from a seven-class taxonomy, at a single share size of 96 by 96 pixels; transfer across sizes was not measured. The clean class is uniform random by construction, so a legitimately re-encoded share is classified as tampered — intended conservative behaviour, but a real constraint. All latency figures come from one development machine under light load, establishing relative cost between stages rather than absolute performance on deployment hardware.

Two narrower defects remain, and two that this report previously listed have since been closed. The laboratory endpoints are still unauthenticated, capped at twenty requests per address per minute by an in-process counter that resets on restart and does not coordinate across workers. And the database is not encrypted at rest: password hashes are Argon2id and message bodies are end-to-end encrypted, so the exposure is smaller than it first appears, but the second-factor secrets are stored in a recoverable form.

The two that were closed are recorded because the reasoning applies to the next such decision. Every dependency is now pinned exactly with a lockfile, after the floor pins were found to have drifted across a major version; and the four flows that collected secrets through window.prompt() now use a masked in-app dialog, guarded by a rule test that fails the build if a browser dialog reappears.

6.4 Future Recommendations

The recommendations are ordered by the ratio of security value to implementation cost, which is not the order of technical interest.

- **1. Move the visual cryptography split into the browser.** This is the single change that would most strengthen the system’s headline claim, converting a partial image guarantee into a full one and removing the server from the trusted computing base for image content entirely. The share generator is 91 lines and the browser already performs pixel-level work for reconstruction.
- **2. Refresh the vendored in-browser runtime.** onnxruntime-web is pinned at 1.19.2 while the Python onnxruntime that validates the export is on 1.31. The export opset has to stay inside what the browser bundle supports, so the two should be moved together and the parity fixture re-run.

- **4. Introduce forward secrecy.** A Double Ratchet construction [19] with an X3DH handshake [20] is the established answer. It is a substantial protocol change and should be scoped as its own project rather than as an increment.
- **5. Extend the feature set against adaptive attacks.** Section 5.6.6 establishes that changing the learner is wasted effort, so the work belongs in the representation. Features sensitive to structure that global statistics cannot see — local randomness tests applied per block, or a comparison of block-level entropy against its expected distribution — are the natural direction. Any such work should be evaluated against an adversary who knows the new features.
- **6. Complete the enhancement objective honestly framed.** As Section 5.6.7 sets out, a residual CNN trained on the matched damaged and clean pairs the attack simulator already produces would be repair of a tampered share, not denoising of a clean reconstruction. It would reuse the validated browser inference path.
- **7. Commission independent security testing.** The cheapest way to find the defects a team cannot see in its own design.
- **8. Authenticate or sandbox the laboratory endpoints.** And move the rate-limiting counters to shared storage so that they survive a restart and coordinate across workers.

6.5 Chapter Summary

Reveal-X delivers nine of its ten objectives and demonstrates that information-theoretically secure image sharing can be embedded in a real-time messaging application at a cryptographic cost measured in tens of microseconds. Its contribution is the integrity layer above an established scheme: a keyed tag and a learned detector that provably cover different failures, with the boundary between them measured rather than assumed. The clearest single result is a negative one — that a classifier over departures from randomness cannot detect an attack that injects randomness, that no learner over those features can, and that this is precisely why the cryptographic layer is not optional.

The system is weaker than production messengers in two respects it states plainly, and its first-phase documentation was inaccurate in ways this report corrects. What it offers in exchange is an account of its own limits that is as precise as its account of its results.

References

References are cited in the text by bracketed number in order of first appearance, following IEEE style.

[1] M. Naor and A. Shamir, "Visual Cryptography," in Advances in Cryptology — EUROCRYPT 1994, Lecture Notes in Computer Science, vol. 950. Berlin: Springer, 1995, pp. 1–12.

[2] A. Shamir, "How to Share a Secret," Communications of the ACM, vol. 22, no. 11, pp. 612–613, Nov. 1979.

[3] G. R. Blakley, "Safeguarding Cryptographic Keys," in Proc. AFIPS National Computer Conference, vol. 48, 1979, pp. 313–317.

[4] G. Ateniese, C. Blundo, A. De Santis and D. R. Stinson, "Visual Cryptography for General Access Structures," Information and Computation, vol. 129, no. 2, pp. 86–106, 1996.

[5] P. Tuyls, H. D. L. Hollmann, J. H. van Lint and L. Tolhuizen, "XOR-based Visual Cryptography Schemes," Designs, Codes and Cryptography, vol. 37, no. 1, pp. 169–186, 2005.

[6] Z. Zhou, G. R. Arce and G. Di Crescenzo, "Halftone Visual Cryptography," IEEE Transactions on Image Processing, vol. 15, no. 8, pp. 2441–2453, Aug. 2006.

[7] Z. Wang, G. R. Arce and G. Di Crescenzo, "Halftone Visual Cryptography via Error Diffusion," IEEE Transactions on Information Forensics and Security, vol. 4, no. 3, pp. 383–396, Sep. 2009.

[8] C. E. Shannon, "A Mathematical Theory of Communication," Bell System Technical Journal, vol. 27, no. 3, pp. 379–423, Jul. 1948.

[9] C. E. Shannon, "Communication Theory of Secrecy Systems," Bell System Technical Journal, vol. 28, no. 4, pp. 656–715, Oct. 1949.

[10] W. Diffie and M. E. Hellman, "New Directions in Cryptography," IEEE Transactions on Information Theory, vol. IT-22, no. 6, pp. 644–654, Nov. 1976.

[11] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Internet Engineering Task Force, Feb. 1997.

[12] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, Internet Engineering Task Force, May 2010.

[13] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001.

[14] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[15] E. Barker, L. Chen, A. Roginsky, A. Vassilev and R. Davis, "Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography," NIST Special Publication 800-56A Rev. 3, Apr. 2018.

[16] D. M'Raihi, S. Machani, M. Pei and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," RFC 6238, Internet Engineering Task Force, May 2011.

[17] A. Biryukov, D. Dinu and D. Khovratovich, "Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications," in Proc. IEEE European Symposium on Security and Privacy (EuroS&P), 2016, pp. 292–302.

[18] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force, Aug. 2018.

[19] M. Marlinspike and T. Perrin, "The Double Ratchet Algorithm," Signal Technical Documentation, Nov. 2016.

[20] M. Marlinspike and T. Perrin, "The X3DH Key Agreement Protocol," Signal Technical Documentation, Nov. 2016.

[21] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[22] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[23] C. R. Harris et al., "Array Programming with NumPy," Nature, vol. 585, pp. 357–362, Sep. 2020.

[24] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120–125, 2000.

[25] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600–612, Apr. 2004.

[26] T. Fawcett, "An Introduction to ROC Analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, Jun. 2006.

[27] J. Fridrich and J. Kodovsky, "Rich Models for Steganalysis of Digital Images," IEEE Transactions on Information Forensics and Security, vol. 7, no. 3, pp. 868–882, Jun. 2012.

[28] B. Bayar and M. C. Stamm, "A Deep Learning Approach to Universal Image Manipulation Detection Using a New Convolutional Layer," in Proc. ACM Workshop on Information Hiding and Multimedia Security (IH&MMSec), 2016, pp. 5–10.

[29] D. Cozzolino and L. Verdoliva, "Noiseprint: A CNN-Based Camera Model Fingerprint," IEEE Transactions on Information Forensics and Security, vol. 15, pp. 144–159, 2020.

[30] K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising," IEEE Transactions on Image Processing, vol. 26, no. 7, pp. 3142–3155, Jul. 2017.

[31] C. Ledig et al., "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 4681–4690.

[32] X. Wang et al., "ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks," in Proc. European Conference on Computer Vision (ECCV) Workshops, 2018, pp. 63–79.

[33] World Wide Web Consortium, "Web Cryptography API," W3C Recommendation, Jan. 2017.

[34] ONNX Community, "Open Neural Network Exchange (ONNX): Intermediate Representation for Machine Learning Models," Specification v1.15, 2023.

[35] Open Web Application Security Project, "OWASP Top 10: The Ten Most Critical Web Application Security Risks," 2021.

[36] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018.

[37] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. Pearson Education, 2018.

[38] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.

[39] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed. Pearson Education, 2020.

[40] X. Wang et al., "Deep Learning Based Image Enhancement for Visual Cryptography Systems," IEEE Access, 2020.

[41] Open Web Application Security Project, "XSS Filter Evasion Cheat Sheet," OWASP Foundation, 2023. [Online]. Available: https://owasp.org/www-community/xss-filter-evasion-cheatsheet

Appendix A: Installation and Interface Reference

Appendix A: Installation and Interface Reference

A.1 Installation

The application runs from a checkout with no build step. On Windows:

```
git clone https://github.com/1337PAIN/Reveal-X-chat.git
cd Reveal-X-chat
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.lock   # the exact tested tree
python run.py
```

The chat is then served at http://127.0.0.1:5000 and the evaluation dashboard, which needs no account, at http://127.0.0.1:5000/lab. Before a demonstration, set REVEAL_X_SECRET_KEY and REVEALX_ADMIN_USER together with REVEALX_ADMIN_PASSWORD; the consequence of leaving the first unset is described in Section 5.2. The server does not reload on code changes unless FLASK_DEBUG is set, and testing against a stale server is an easy way to misdiagnose a fault.

To run the verification suites:

```
pytest                              # 416 server-side tests
node --test "tests/js/*.test.mjs"   # 69 browser-side tests
```

Two of the server tests are skipped unless a Firebase service account is configured. A secure context is required for a demonstration on a phone: without HTTPS, the Web Crypto API, the service worker and microphone access are all unavailable, so end-to-end encryption, installation and voice notes will each appear broken.

A.2 HTTP routes

Table A.1: The nine HTTP routes.

Route	Authentication	Purpose
/	None	Chat application
/lab	None	Evaluation dashboard
/lab/parity	None	Browser-versus-server feature parity evidence
/api/auth/config	None	Which sign-in methods this deployment offers
/api/lab/features	None	Feature extraction; 20 requests per address per minute
/api/lab/process	None	Full pipeline; shares the same budget
/api/share/<id>/analysis	Session	Verify and classify a stored Share 1 without consuming access
/share1/<id>?token=…	Capability token	Serve Share 1, consuming the one-time access
/sw.js	None	Service worker, served at root scope

A.3 Socket.IO events

The thirty-two real-time events, grouped by concern.

Table A.2: Socket.IO event inventory.

Group	Events
Connection	connect, disconnect
Authentication	register, login, firebase_login, verify_totp, logout
Second factor	totp_begin_enrol, totp_confirm_enrol, totp_disable
Keys	register_public_key
Profile and settings	update_profile, update_username, update_bio, update_status, delete_account
Messaging	select_chat, send_message, send_voice, send_image, react_message, forward_message, mark_read, typing
Message lifecycle	revoke_share, delete_message
Administration	admin_list_accounts, admin_set_state, admin_create_account, admin_reset_password, admin_delete_account, admin_set_role

A.4 Repository layout

Table A.3: Principal source files.

Path	Contents
app/routes.py	HTTP routes, socket handlers, throttling, share endpoints
app/models.py	Storage layer, schema, retention, the Share 2 refusal
app/vc_core/	Share generation and reconstruction
app/ai_security/	Tamper detector, HMAC integrity, metrics, attack simulator, enhancement
app/auth/	Firebase ID token verification and TOTP
app/static/js/	Chat, E2EE, in-browser ML, sanitisation, service worker
app/static/models/	tamper_detector.onnx, 40.2 KB
scripts/	Model training, ONNX export, parity fixture generation, migration
tests/	416 Python tests; tests/js holds the 69 browser tests
docs/	Threat model, API reference, deployment, administration, accessibility